

# NOMAD - Navigating Optimal Model Application to Datastreams

Ashwin Gerard Colaco  
University of California, Irvine

Sharad Mehrotra  
University of California, Irvine

Michael J De Lucia  
USARMY DEVCOM ARL

Kevin Hamlen  
The University of Texas at Dallas

Murat Kantarcioglu  
Virginia Tech

Latifur Khan  
The University of Texas at Dallas

Ananthram Swami  
USARMY DEVCOM ARL

Bhavani Thuraisingham  
The University of Texas at Dallas

## ABSTRACT

NOMAD (Navigating Optimal Model Application for Data-streams) is an intelligent framework for data enrichment during ingestion that optimizes real-time multiclass classification by dynamically constructing model chains—sequences of machine learning models with varying cost-quality tradeoffs, selected via a utility-based criterion. Inspired by predicate-ordering techniques from database query processing, NOMAD leverages cheaper models as initial filters, proceeding to more expensive models only when necessary, while guaranteeing classification quality remains  $\epsilon$ -comparable to a designated role model through a formal chain safety mechanism. It employs a dynamic belief update strategy to adapt model selection based on per-event predictions and shifting data distributions, and extends to scenarios with dependent models such as early-exit DNNs and stacking ensembles. Evaluation across multiple datasets demonstrates that NOMAD achieves significant computational savings (speedups of 2–6 $\times$ ) compared to static and naive approaches while maintaining classification quality comparable to that achieved by the most accurate (and often the most expensive) model.

### PVLDB Reference Format:

Ashwin Gerard Colaco, Sharad Mehrotra, Michael J De Lucia, Kevin Hamlen, Murat Kantarcioglu, Latifur Khan, Ananthram Swami, and Bhavani Thuraisingham. NOMAD - Navigating Optimal Model Application to Datastreams. PVLDB, 14(1): XXX-XXX, 2020. doi:XX.XX/XXX.XX

### PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at URL\_TO\_YOUR\_ARTIFACTS.

## 1 INTRODUCTION

Modern data processing systems increasingly require data ingestion components capable of interpreting raw data in real-time. Data ingestion—the process of continuously collecting, importing, and processing data as it arrives—often becomes a critical performance bottleneck [11, 33]. This process involves extracting features, applying trained models, and executing actions based on model outcomes, all while data streams into the system at high rates.

Applications of such real-time ingestion are widespread. For instance, consider a cybersecurity use case: the input data consists of network traffic arriving as a stream, and the goal is to classify each flow into categories such as benign activity or different types of attacks—for example, Distributed Denial of Service (DDoS), port hopping, or more sophisticated intrusion techniques [31]. Such classification may invoke appropriate actions, including triggering countermeasures, tagging the information for database storage, or escalating alerts to cybersecurity analysts. Another example involves dynamic video data captured from live cameras. Ingesting such data may involve running models to classify the video and tag segments based on a given event/entity taxonomy (e.g., start/end of a meeting, appearance of a person or a vehicle, etc.). Object detection models such as YOLO [25], or ResNet [13] can be used to detect and classify objects in footage, while embedding models like CLIP [23] can enable zero-shot classification by comparing image and text representations. Such models can detect attributes like room occupancy or tag segments with specific labels. Such tags facilitate information triage, user-specific distribution, and enable advanced search and analysis capabilities.

These ingestion tasks often correspond to multiclass classification. In the examples above, models like Decision Tree classifiers, Support Vector Machines (SVM), XGBoost, Random Forest, deep neural networks (DNN), CLIP, YOLO, and ResNet can be trained or utilized to differentiate between multiple classes. Such models often exhibit a cost-quality tradeoff. For instance, decision trees are computationally efficient but may not offer the highest accuracy. In contrast, random forests generally perform well for complex tasks but have significantly higher computational costs. Similarly, neural networks can achieve high accuracy, but their performance depends heavily on factors like architecture, size, and data distribution.

A naive strategy is to always run the most accurate model to ensure high-quality results. However, such an approach makes ingestion unnecessarily expensive when cheaper models could produce identical classifications for much of the input data. The challenge lies in achieving quality comparable to the best model while substantially reducing cost.

This paper presents NOMAD, an intelligent ingestion architecture designed to enable efficient, high-quality multiclass classification to address this challenge. NOMAD draws inspiration from cascade classifiers [32] which sequences increasingly complex classifiers to quickly reject easy negative cases while reserving expensive computation for hard examples. Transitioning from binary detection in [32] to multiclass setting introduces fundamental complexity: the choice is no longer whether to stop or to invoke another

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing [info@vldb.org](mailto:info@vldb.org). Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.  
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.  
doi:XX.XX/XXX.XX

(likely more accurate but more expensive) model based on the confidence of the previous model, but also which model to invoke further based on the predictions of prior models. While prior work has explored multiclass cascades (e.g., cost-sensitive tree-structured cascades [36], sequential classification under budget constraints [28], and multiclass boosting cascades [26]), existing work exhibit significant shortcomings when used in dynamic ingestion settings. Existing work has focused on learning a fixed policy of invoking different models at training time. Static policies cannot adapt to distribution shifts common in ingestion streams (evolving attacks, seasonal patterns), nor can they adapt to addition/removal of new models. Furthermore, much of existing work impose model-specific constraints (e.g., boosting cascades [26] require weak learners; tree cascades [36] learn fixed topologies), preventing easy incorporation of heterogeneous pretrained models. Finally, existing approaches do not provide any quality guarantees relative to a designated high-quality model – essential for downstream processing that depends on reliable classifications.

NOMAD addresses these limitations by: (1) providing provable guarantee that the quality of classification achieved will be close to that achievable by the best model in the cascade, (2) dynamically adapting model selections using lightweight statistical techniques (ARIMA, Page-Hinkley) that adjust to distribution shifts without expensive retraining; (3) supporting a model-agnostic design accepting heterogeneous pretrained models as black boxes; and (4) supporting a utility-based selection with dynamic belief updates that navigates the space of model sequences by leveraging per-class accuracy patterns rather than learning fixed cascade structures.

The core idea in NOMAD involves composing models into optimized sequences or decision graphs (model chains) based on expected class distributions and class-specific criteria. NOMAD leverages each model’s class-specific performance to determine whether to accept its prediction or proceed to another model in the chain. As the system processes data, NOMAD dynamically selects which models to chain based on ongoing predictions and expected distributions. The framework accommodates both atomic ML models (such as Random Forest, XGBoost, CART, and LDA) and more complex compositions of atomic models. Such complex models may require (possibly multiple) submodels to execute prior to their execution. Examples of such model compositions include ensemble stacking methods that may require component models to be executed before execution of the stacker, and early-exit deep neural networks where invoking the classification at one layer implicitly requires executing prior layers in the network. For ease of exposition, we initially develop NOMAD assuming models operate independently and are invoked on a single event at a time. We then extend NOMAD to scenarios when models used may be dependent where execution follows prerequisite relationships (e.g., early exit neural nets, ensemble methods), and, furthermore, when model invocation is batched amortizing the time overhead of model reloading. We evaluate NOMAD on multiple datasets, demonstrating significant cost savings without compromising classification quality compared to static and naive approaches.

Our contributions include: (1) formalization of the Model Selection Problem with quality guarantees; (2) *chain safety*, characterizing when model chains meet quality guarantees; (3) utility-based model selection with dynamic belief updates to optimize cost while

| Symbol                              | Description                         |
|-------------------------------------|-------------------------------------|
| $E$                                 | Stream of events                    |
| $C = \{C_1, \dots, C_k\}$           | Set of $k$ classes                  |
| $\mathcal{M} = \{M_1, \dots, M_n\}$ | Set of $n$ models                   |
| $M_r$                               | Role model (quality benchmark)      |
| $\text{cost}(M_i)$                  | Inference cost of model $M_i$       |
| $D_{val}$                           | Validation dataset                  |
| $CM^{(i)}$                          | Confusion matrix for model $M_i$    |
| $Q(M, C_j)$                         | Quality of model $M$ on class $C_j$ |
| $\epsilon$                          | Quality tolerance factor            |
| $EC(M_i)$                           | Exit classes for model $M_i$        |
| $S$                                 | Classification strategy             |
| $S_e$                               | Realized model chain for event $e$  |
| $Prob(C_j)$                         | Probability of class $C_j$          |

**Table 1: Key notation used in this paper.**

ensuring quality guarantees; (4) extensions of NOMAD; (5) lightweight adaptation using ARIMA and Page-Hinkley drift detection; and (6) experimental validation on eight datasets demonstrating 2–6 $\times$  speedups while maintaining quality guarantees.

Section 2 defines the model selection problem. Sections 3 and 4 present NOMAD and its guarantees. Sections 5 and 6 discuss extensions to dependent models & batched execution, and distribution shifts. Section 7 presents experimental results, Section 8 covers related work, and Section 9 concludes.

## 2 PROBLEM FORMALIZATION

Consider a data stream of events  $E = \{e_1, e_2, e_3, \dots\}$  that are continuously ingested. The objective is to classify each incoming event  $e \in E$  into one of several predefined classes  $C = \{C_1, C_2, \dots, C_k\}$  as quickly and accurately as possible. This real-time classification task must balance computational cost and classification quality.

To achieve this, a set of models  $\mathcal{M} = \{M_1, M_2, \dots, M_n\}$  is available. Each model  $M_i$  has an associated classification cost,  $\text{cost}(M_i)$ , and a measure of its classification quality. Without loss of generality, we assume the models are indexed according to their non-decreasing costs:  $\text{cost}(M_1) \leq \text{cost}(M_2) \leq \dots \leq \text{cost}(M_n)$ .

In practice, the input workload follows an expected distribution over the classes. We denote the probability of encountering an event of class  $C_j$  as  $Prob(C_j)$ , where  $\sum_{j=1}^k Prob(C_j) = 1$ . This distribution characterizes the typical composition of the event stream.

In the context of the network intrusion detection example from the Introduction, the classes  $C$  could represent categories such as normal traffic ( $C_1$ ), denial-of-service (DoS) attacks ( $C_2$ ), phishing attempts ( $C_3$ ), and malware transmissions ( $C_4$ ).

Table 1 summarizes the key notation used throughout this paper.

### 2.1 Measure of Model Quality

The performance of each individual model is characterized on a static, representative validation dataset, denoted  $D_{val}$ . For a system with  $k$  classes, each model  $M_i$  produces a  $k \times k$  confusion matrix,  $CM^{(i)}$ , from its predictions on  $D_{val}$ . The element  $CM_{jl}^{(i)}$  represents the number of events with true class  $C_j$  that are predicted as class  $C_l$  by model  $M_i$ .

|                                                                                                                                                                  |                                                                                                                                                                  |                                                                                                                                                                  |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $\begin{bmatrix} 0.900 & 0.050 & 0.030 & 0.020 \\ 0.060 & 0.800 & 0.080 & 0.060 \\ 0.100 & 0.070 & 0.730 & 0.100 \\ 0.030 & 0.040 & 0.050 & 0.880 \end{bmatrix}$ | $\begin{bmatrix} 0.950 & 0.030 & 0.010 & 0.010 \\ 0.040 & 0.900 & 0.040 & 0.020 \\ 0.020 & 0.030 & 0.920 & 0.030 \\ 0.015 & 0.025 & 0.030 & 0.930 \end{bmatrix}$ | $\begin{bmatrix} 0.980 & 0.010 & 0.005 & 0.005 \\ 0.015 & 0.960 & 0.015 & 0.010 \\ 0.010 & 0.020 & 0.960 & 0.010 \\ 0.005 & 0.015 & 0.010 & 0.970 \end{bmatrix}$ |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|

**Table 2: Confusion matrices for classifiers  $M_1$ ,  $M_2$ , and  $M_3$ .**

**Example 2.1.** Consider three classifiers evaluated on the same  $D_{val}$ . The models may correspond to **Classifier  $M_1$**  (low-cost), **Classifier  $M_2$**  (medium-cost), and **Classifier  $M_3$**  (high-cost). Table 2 shows their confusion matrices, where each entry  $CM_{jl}^{(i)}$  represents the probability that model  $M_i$  predicts class  $C_l$  when the true class is  $C_j$  (i.e.,  $CM_{jl}^{(i)}$  is the count of such events divided by the total number of class  $C_j$  events in  $D_{val}$ ).

From a model’s confusion matrix, we can derive various quality metrics such as precision, recall, F1-score, or accuracy. Such quality measures can be defined at the level of individual classes or globally across all classes. We use the notation  $Q(M, C)$  for a model  $M$  evaluated on class  $C$  for class-based metrics, and  $Q(M)$  for global metrics which is a macro average over class based quality. Our choice of quality metric is discussed in Section 7.

## 2.2 Problem Definition

We now formalize the problem of cost-efficient classification.

**Definition 2.1 (Role Model).** A role model  $M_r \in \mathcal{M}$  is designated as the benchmark for classification quality.  $M_r$  is typically the model with the highest  $Q(M_r, C_j)$  values across all classes, often corresponding to the most expensive model.

**Definition 2.2 (Class-based  $\epsilon$ -Comparability).** Given two models  $M_i, M_j \in \mathcal{M}$  trained on a given dataset,  $M_i$  provides  $\epsilon$ -comparable quality to  $M_j$  for a class  $C_k \in \mathcal{C}$  if its pre-computed quality is within a tolerance factor  $(1 - \epsilon)$  of  $M_j$ ’s quality:

$$Q(M_i, C_k) \geq Q(M_j, C_k) \times (1 - \epsilon) \quad (1)$$

Here,  $\epsilon \in [0, 1]$  is the  $Q$  tolerance factor and is the maximum acceptable fractional degradation in quality.

**Definition 2.3 (Global  $\epsilon$ -Comparability).** A model  $M_i$  provides global  $\epsilon$ -comparable quality to model  $M_j$  if its weighted average quality across all classes is comparable:

$$\sum_{k=1}^{|\mathcal{C}|} \text{Prob}(C_k) \cdot Q(M_i, C_k) \geq (1 - \epsilon) \cdot \sum_{k=1}^{|\mathcal{C}|} \text{Prob}(C_k) \cdot Q(M_j, C_k) \quad (2)$$

where  $\text{Prob}(C_k)$  is the probability of  $C_k$  in the expected distribution.

Global  $\epsilon$ -comparability is weaker than class-based  $\epsilon$ -comparability - viz., a model satisfying class-based requirement for every class always satisfies the global one, but the converse is not true as we will show in 4.

**Definition 2.4 (Exit Model for a Class).** Let  $M_r$  be the role model and  $\epsilon$  be the quality tolerance factor. A model  $M_i \in \mathcal{M}$  is an *exit model* for class  $C_j$  if it provides class-based  $\epsilon$ -comparable quality to  $M_r$  for that class.

Note that according to this definition, multiple models (including  $M_r$  itself) can be exit models for the same class  $C_j$ .

| Strategy A: $CM^{(S_A)}$                                                                                                                                         | Strategy B: $CM^{(S_B)}$                                                                                                                                         |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $\begin{bmatrix} 0.976 & 0.004 & 0.000 & 0.020 \\ 0.100 & 0.810 & 0.002 & 0.088 \\ 0.080 & 0.034 & 0.812 & 0.074 \\ 0.033 & 0.003 & 0.000 & 0.964 \end{bmatrix}$ | $\begin{bmatrix} 0.950 & 0.008 & 0.002 & 0.040 \\ 0.086 & 0.864 & 0.014 & 0.036 \\ 0.078 & 0.016 & 0.870 & 0.036 \\ 0.022 & 0.004 & 0.006 & 0.968 \end{bmatrix}$ |

**Table 3: Resulting confusion matrices for  $S_A$  and  $S_B$ .**

**Definition 2.5 (Exit Classes for a Model).** For a model  $M_i \in \mathcal{M}$ , its set of *exit classes*,  $EC(M_i)$ , are those for which  $M_i$  provides class-based  $\epsilon$ -comparable quality to the role model  $M_r$ :

$$EC(M_i) = \{C_j \in \mathcal{C} \mid Q(M_i, C_j) \geq Q(M_r, C_j) \times (1 - \epsilon)\} \quad (3)$$

By definition,  $EC(M_r) = \mathcal{C}$  for any  $\epsilon \geq 0$ .

Now we define the dynamic components of our system.

**Definition 2.6 (Classification Strategy).** A *Classification Strategy*,  $\mathcal{S}$ , defines the dynamic process for selecting and executing one or more models from  $\mathcal{M}$  to classify an incoming event.

**Definition 2.7 (Realized Model Chain for an Event).** For an event  $e$ , the *Realized Model Chain*,  $S_e = (M^{(1)}, \dots, M^{(\ell)})$ , is the sequence of models executed by strategy  $\mathcal{S}$  until a prediction falls into an exit class. Here,  $M^{(i)}$  denotes the  $i$ -th model executed in the chain for event  $e$ , and  $\ell$  is the total number of models executed. Let  $\text{Out}(M, e)$  be the predicted class when model  $M$  processes event  $e$ . The chain terminates at  $M^{(\ell)}$  where:

- $\text{Out}(M^{(i)}, e) \notin EC(M^{(i)})$  for all  $i < \ell$ .
- $\text{Out}(M^{(\ell)}, e) \in EC(M^{(\ell)})$ .

The final classification for the event is  $\text{Out}(\mathcal{S}, e) = \text{Out}(M^{(\ell)}, e)$ .

Given a strategy  $\mathcal{S}$ , we denote its quality as  $Q(\mathcal{S}, C_j)$ , which can be computed by simulating the execution of the strategy and evaluating its performance over the validation dataset  $D_{val}$ .

**Example 2.2.** Consider the following two strategies:  $S_A$  and  $S_B$  based on the models in Example 2.1, with  $M_3$  acting as the high-quality role model. **Strategy A:** A full chain  $S_A = (M_1 \rightarrow M_2 \rightarrow M_3)$ . **Strategy B:** A shorter chain  $S_B = (M_1 \rightarrow M_3)$ . Suppose the exit classes for models  $M_1$  and  $M_2$  are as follows:  $EC(M_1) = \{C_1, C_4\}$  and  $EC(M_2) = \{C_1, C_2, C_4\}$ .

To determine quality of a strategy  $S_A$ , each event  $e$  in  $D_{val}$  is first processed by  $M_1$ . If  $M_1$  predicts a class in  $EC(M_1)$ , the chain terminates with the class predicted by  $M_1$  as the outcome of  $S_A$ . Else,  $e$  is passed to  $M_2$ , and, as before, if the prediction of  $M_2$  is in  $EC(M_2)$ , that becomes the prediction of  $S_A$ . The process continues until the last model in the chain has executed. By executing  $S_A$  on all events in  $D_{val}$  and recording the final predictions versus true labels, we can construct a confusion matrix for  $S_A$ . Table 3 shows plausible confusion matrices for the strategies  $S_A$  and  $S_B$  over validation set  $D_{val}$ .

Based on these confusion matrices, the quality of the strategy can be determined. For instance, the class-based recall for  $S_A$  and  $S_B$  for the class  $C_1$  are 0.976 and 0.950 and their overall recall is 0.891 and 0.913 respectively.

In addition to quality, we also need to consider the computational cost of a strategy. Since different events may follow different

paths through the model chain (depending on which models' exit conditions are met), the cost varies by event.

**Definition 2.8 (Strategy Cost).** For a strategy  $\mathcal{S}$  and its realized model chain  $S = (M^{(1)}, \dots, M^{(\ell)})$  for a particular event, the **cost of the strategy** is the sum of the costs of all models executed:

$$\text{cost}(S) = \sum_{i=1}^{\ell} \text{cost}(M^{(i)}) \quad (4)$$

where  $\ell$  is the length of the realized chain, and  $M^{(i)}$  is the  $i$ -th model executed in that chain.

Note that the realized chain, and thus the cost, depends on the specific event being classified. Different events may terminate at different models based on the exit conditions, resulting in variable costs per event across the event stream.

We can now formally define the model selection problem.

**Definition 2.9 (Model Selection Problem (MSP)).** Given a set of models  $\mathcal{M}$ , a role model  $M_r$ , and a set of classes  $C$ , expected distribution  $p(C_j)$  for  $j = 1, \dots, |C|$ , and tolerance  $\epsilon$ , the *Model Selection Problem* is to find a **Classification Strategy  $\mathcal{S}$**  that minimizes average cost  $\mathbb{E}[\text{cost}(S)]$  (expectation over realized chains  $S$  for events drawn from the expected distribution) while ensuring the strategy's quality is  $\epsilon$ -comparable to the role model's quality.

The quality constraint in the model selection problem can be formulated in two ways based on the notion of quality:

- (1) **Class-specific Quality Requirement:** For every class, the strategy must maintain quality comparable to the role model:

$$\forall C_j \in C, \quad Q(\mathcal{S}, C_j) \geq Q(M_r, C_j) \times (1 - \epsilon)$$

- (2) **Global Quality Requirement:** The strategy's weighted average quality must be comparable to the role model:

$$\sum_{j=1}^{|C|} \text{Prob}(C_j) \cdot Q(\mathcal{S}, C_j) \geq (1 - \epsilon) \cdot \sum_{j=1}^{|C|} \text{Prob}(C_j) \cdot Q(M_r, C_j)$$

The class-specific constraint is more stringent, guaranteeing performance for every class including rare ones. The global constraint allows trading off performance across classes, potentially achieving lower costs but risking poor performance on minority classes.

Note that the MSP always has a trivial feasible solution: a strategy that exclusively uses the role model  $M_r$  for all events. This satisfies any quality constraint (with  $Q(\mathcal{S}, C_j) = Q(M_r, C_j)$ ) but incurs the maximum possible cost. The challenge lies in designing strategies that leverage cheaper models when appropriate while provably maintaining the required quality level.

In the following sections, we present NOMAD, our solution to the Model Selection Problem. NOMAD dynamically constructs model chains that minimize cost while ensuring the quality constraints are satisfied through a combination of strategic model selection, safety checks, and adaptive belief updates.

### 3 NOMAD ALGORITHM

NOMAD classifies streaming input events while balancing computational cost and classification quality, ensuring final predictions meet a specified quality guarantee relative to role model  $M_r$ . It

processes each event through an iterative procedure that constructs a sequence of one or more models to derive a classification. Figure 1 depicts this architecture.

The algorithm requires two types of prior information: a probability distribution over classes  $C = \{C_1, \dots, C_k\}$ , denoted  $\text{Prob}(C_j)$  where  $\sum_{j=1}^k \text{Prob}(C_j) = 1$  (representing initial class expectations, assumed known for now; Section 6 discusses learning this distribution), and for each model  $M_i \in \mathcal{M}$ , pre-computed quality metrics  $Q(M_i, C_j)$ , cost, and exit classes  $\text{EC}(M_i)$ .

Algorithm 1 outlines single-event processing. The process begins with initial beliefs about the event's class (line 1) and enters a loop (lines 2-11) that iteratively selects models, checks safety, and updates beliefs. Each iteration selects the highest-utility model from available models (line 3). If selected and safe (line 5), it executes the model (line 6) and appends to the chain (line 7). If prediction falls into an exit class (line 8), the process terminates; otherwise, beliefs are updated using the model's softmax output (line 10) and the loop continues with remaining models (line 11). If no safe chain exists, the algorithm falls back to  $M_r$  (line 12). The key functions 'SelectNextModel' and 'UpdateBeliefs' are detailed below; 'CheckChainSafety', critical for quality guarantees, is detailed in the next section.

#### 3.1 Utility-based Model Selection

The 'SelectNextModel' function selects the model with highest expected utility, defined as the ratio of effectiveness to cost, where effectiveness equals the sum of exit class probabilities based on current belief state  $p_{\text{current}}$ . The utility score  $U(M_i)$  for model  $M_i$  given current belief vector  $p_{\text{current}}$  is:

$$U(M_i) = \frac{\sum_{C_j \in \text{EC}(M_i)} \text{Prob}_{\text{current}}(C_j)}{\text{cost}(M_i)}$$

This captures the trade-off between a model's applicability to the current event and its resource consumption. The selection iterates through available models (lines 2-8), computing each model's exit probability (line 3) and utility score (line 4), tracking maximum utility (lines 5-7), and returning the best model (line 9).

#### 3.2 Belief Update

If executed model  $M^*$  returns prediction  $C_{\text{pred}} \notin \text{EC}(M^*)$ , the system updates its belief state using 'UpdateBeliefs', which refines the class probability distribution by incorporating the model's softmax output vector  $s$ .

The update is a Bayesian operation[18] using element-wise product (Hadamard product) of current probability vector  $p_{\text{current}}$  and softmax vector  $s$ , followed by re-normalization. Non-exit predictions can rule out classes: if a model's classifications for its exit classes are reliable, a non-exit prediction implies the true class is not among those exit classes. The algorithm supports this by zeroing out "ruled-out" classes before the main update. Specifically, the algorithm zeros out probabilities for ruled-out classes (lines 2-3), performs the Hadamard product (line 4), computes the normalization constant (line 5), and normalizes to obtain updated beliefs (lines 6-9). If the normalization constant is too small (indicating numerical instability), it falls back to uniform distribution (line 9). This process is formalized in Algorithm 3.

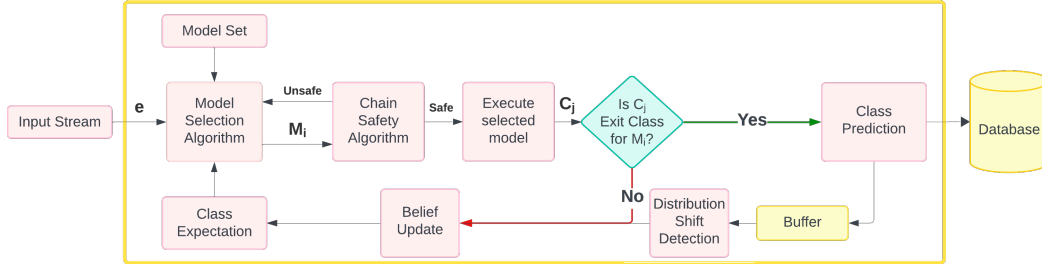


Figure 1: NOMAD's Data Ingestion Framework

#### Algorithm 1: NOMAD Event Processing

```

1 Function NOMAD_ProcessEvent( $e, \mathcal{M}, M_r, \text{InitialBeliefs}, \epsilon$ )
2    $\mathcal{M}_{\text{available}} \leftarrow \mathcal{M}; \text{Prob}_{\text{current}} \leftarrow \text{InitialBeliefs};$ 
3    $S_{\text{current}} \leftarrow \emptyset;$  // The current model chain
4   while  $\mathcal{M}_{\text{available}} \neq \emptyset$  do
5      $M_{\text{selected}} \leftarrow \text{SelectNextModel}(\text{Prob}_{\text{current}}, \mathcal{M}_{\text{available}});$ 
6     if  $M_{\text{selected}}$  is null then break;
7     if CheckChainSafety( $M_{\text{selected}}, S_{\text{current}}, M_r, \epsilon, C$ ) then
8        $(C_{\text{pred}}, s) \leftarrow \text{Execute}(M_{\text{selected}}, e);$ 
9        $S_{\text{current}} \leftarrow S_{\text{current}} \circ (M_{\text{selected}});$ 
10      // Append to chain
11      if  $C_{\text{pred}} \in \text{EC}(M_{\text{selected}})$  then return  $C_{\text{pred}};$ 
12      // Exit condition met
13      else  $\text{Prob}_{\text{current}} \leftarrow$ 
14         $\text{UpdateBeliefs}(\text{Prob}_{\text{current}}, s, \text{EC}(M_{\text{selected}}));$ 
15       $\mathcal{M}_{\text{available}} \leftarrow \mathcal{M}_{\text{available}} \setminus \{M_{\text{selected}}\};$ 
16   $(C_{\text{final}}, \_) \leftarrow \text{Execute}(M_r, e);$  // Fallback to  $M_r$ 
17  return  $C_{\text{final}};$ 

```

#### Algorithm 2: Select Next Model

```

1 Function SelectNextModel( $p_{\text{current}}, \mathcal{M}_{\text{available}}$ )
2    $M_{\text{best}} \leftarrow \text{null}; U_{\text{max}} \leftarrow -1;$ 
3   foreach model  $M_i \in \mathcal{M}_{\text{available}}$  do
4      $\text{Prob}_{\text{exit\_sum}} \leftarrow \sum_{C_j \in \text{EC}(M_i)} \text{Prob}_{\text{current}}(C_j);$ 
5      $U_i \leftarrow \text{Prob}_{\text{exit\_sum}} / \text{cost}(M_i);$ 
6     if  $U_i > U_{\text{max}}$  then
7        $U_{\text{max}} \leftarrow U_i;$ 
8        $M_{\text{best}} \leftarrow M_i;$ 
9   return  $M_{\text{best}};$ 

```

NOMAD dynamically adapts its classification strategy for each event to minimize computational cost while adhering to quality constraints through utility-based model selection, belief updating, and chain safety (formalized next). The framework generalizes for dependent models (Section 5) and uses distribution shift detection to adapt to long-term data stream changes (Section 6).

#### Algorithm 3: Update Beliefs

```

1 Function UpdateBeliefs( $\text{Prob}_{\text{in}}, s, C_{\text{ruled\_out}}$ )
2    $\text{Prob}_{\text{temp}} \leftarrow \text{Prob}_{\text{in}};$ 
3   foreach class  $C_j \in C_{\text{ruled\_out}}$  do
4      $\text{Prob}_{\text{temp}}(C_j) \leftarrow 0;$  // Zero out probabilities
5     // for ruled-out classes
6    $\text{Prob}_{\text{new}} \leftarrow \text{Prob}_{\text{temp}} \odot s;$ 
7   //  $\odot$  is element-wise product
8    $Z \leftarrow \sum_j (\text{Prob}_{\text{new}})_j$  // Compute normalization
9   if  $Z > 10^{-9}$  then
10     $\text{Prob}_{\text{out}} \leftarrow \text{Prob}_{\text{new}} / Z;$ 
11  else
12     $k \leftarrow$  number of classes in  $C;$ 
13     $\text{Prob}_{\text{out}} \leftarrow \{1/k\}_{j=1}^k;$ 
14    // Fallback to uniform distribution
15  return  $\text{Prob}_{\text{out}};$ 

```

## 4 CHAIN SAFETY

Chain safety is a fundamental mechanism in NOMAD that ensures quality guarantees are maintained when models are sequenced together. Intuitively, a chain of models is unsafe if it risks failing to achieve  $\epsilon$ -comparable quality to the role model. This can occur when intermediate models in the chain misclassify events prematurely, preventing those events from reaching a model that would classify them correctly. Chain safety formalizes the conditions under which models can be safely sequenced without this risk.

In this section, we present NOMAD's default chain safety mechanism, which uses a *global* quality guarantee with *relaxed* passthrough estimation. This configuration, used throughout our experiments, provides an effective balance between computational efficiency and quality assurance.

To formalize chain safety, we associate the notions of misclassification probability and passthrough probability with each model. For an event of a given class  $C_j$  that is not an exit class for a model  $M$  (i.e.,  $C_j \notin \text{EC}(M)$ ), the event can be potentially misclassified into one of  $M$ 's exit classes. We refer to the probability of such an occurrence as the misclassification probability,  $\text{Prob}_{mc}(M, C_j)$ . This can be estimated from confusion matrix  $CM^{(i)}$  for model  $M_i$ :

$$\text{Prob}_{mc}(M_i, C_j) = \frac{\sum_{C_k \in \text{EC}(M_i)} CM_{jk}^{(i)}}{N_j}$$

where  $N_j$  is the total number of validation instances of class  $C_j$ . Note that  $\text{Prob}_{mc}(M_i, C_j) \leq 1$  by construction: the numerator  $\sum_{C_k \in \text{EC}(M_i)} CM_{jk}^{(i)}$  counts the total number of class  $C_j$  events misclassified into exit classes, which cannot exceed the total number of class  $C_j$  events,  $N_j$ .

Consequently, the passthrough probability,  $\text{Prob}_{pt}(M, C_j)$ , is the probability that an event of class  $C_j$  is not misclassified into an exit class by model  $M$ , and is thus allowed to “pass through” to the next model in the chain. It is defined as:

$$\text{Prob}_{pt}(M, C_j) = 1 - \text{Prob}_{mc}(M, C_j)$$

For instance, for our models in Example 2.9, the passthrough probability of a class  $C_2$  event through model  $M_1$  is  $\text{Prob}_{pt}(M_1, C_2) = 0.88$ , while for a class  $C_3$  event it is  $\text{Prob}_{pt}(M_1, C_3) = 0.80$ . Likewise, for a  $C_3$  event and model  $M_2$ , it is  $\text{Prob}_{pt}(M_2, C_3) = 0.92$ . Next we show how we compute passthrough probability for a model and then show how we determine if a chain is safe.

#### 4.1 Computing Passthrough Probability

For an event of class  $C_j$  that is not an exit class for model  $M$  (i.e.,  $C_j \notin \text{EC}(M)$ ), the event can potentially be misclassified into one of  $M$ ’s exit classes. We refer to the probability of such an occurrence as the **misclassification probability**,  $\text{Prob}_{mc}(M, C_j)$ .

Consequently, the **passthrough probability**,  $\text{Prob}_{pt}(M, C_j)$ , is the probability that an event of class  $C_j$  is not misclassified into an exit class by model  $M$ , and is thus allowed to “pass through” to the next model in the chain:

$$\text{Prob}_{pt}(M, C_j) = 1 - \text{Prob}_{mc}(M, C_j) \quad (5)$$

We estimate passthrough probabilities using **relaxed estimation with Laplace smoothing**, which uses the entire confusion matrix to provide accurate estimates. For a model  $M_k$  predicting class  $C_p$  given true class  $C_t$ :

$$\text{Prob}_{\text{smoothed}}(C_p|C_t) = \frac{CM_{tp}^{(k)} + \alpha}{N_t + \alpha \cdot |C|} \quad (6)$$

where  $\alpha = 1$  (Laplace smoothing). We sum these probabilities over all exit classes to obtain the misclassification probability, and the passthrough estimate is its complement:  $\text{Prob}_{pt}^{est}(M_i, C_j) = 1 - \sum_{C_k \in \text{EC}(M_i)} \text{Prob}_{\text{smoothed}}(C_k|C_j)$ . With these concepts, we now show how a chain’s safety is determined.

#### 4.2 Global Chain Safety

NOMAD supports both global and class-based safety guarantees. We present the global safety mechanism here (our default configuration, see Section 7). The global chain safety mechanism ensures that a strategy’s expected quality, weighted by the class probabilities  $\text{Prob}(C_j)$ , is  $\epsilon$ -comparable to the role model’s quality. This provides a guarantee on average performance across all classes while allowing flexibility for the system to optimize cost by potentially trading off performance across different classes.

For a potential chain  $\mathcal{S}$  and a class  $C_j$ , the projected quality  $Q_{proj}(\mathcal{S}, C_j)$  is the quality of its designated exit model, discounted by the cumulative probability of the event successfully passing through all preceding models. For a chain to be globally safe, the

weighted sum of projected qualities must be  $\epsilon$ -comparable to the role model:

$$\sum_{j=1}^{|C|} \text{Prob}(C_j) \cdot Q_{proj}(\mathcal{S}, C_j) \geq (1 - \epsilon) \cdot \sum_{j=1}^{|C|} \text{Prob}(C_j) \cdot Q(M_r, C_j) \quad (7)$$

**Example 4.1.** Consider strategies from Example 2.2:  $\mathcal{S}_A = (M_1 \rightarrow M_2 \rightarrow M_3)$  and  $\mathcal{S}_B = (M_1 \rightarrow M_3)$  with  $M_3$  as role model and  $\epsilon = 0.1$ . The class distribution is  $\text{Prob}(C_1) = 0.4$ ,  $\text{Prob}(C_2) = 0.3$ ,  $\text{Prob}(C_3) = 0.2$ , and  $\text{Prob}(C_4) = 0.1$ .

For  $\mathcal{S}_A$ ,  $\text{EC}(M_1) = \{C_1, C_4\}$  and  $\text{EC}(M_2) = \{C_1, C_2, C_4\}$ . From Table 2’s confusion matrices, passthrough probabilities using relaxed estimation are  $\text{Prob}_{pt}(M_1, C_2) = 0.91$ ,  $\text{Prob}_{pt}(M_1, C_3) = 0.85$ , and  $\text{Prob}_{pt}(M_2, C_3) = 0.92$ .

Projected quality for each class discounts the exit model’s quality by cumulative passthrough probability.

For  $C_1$ , which exits at  $M_1$ ,  $Q_{proj}(\mathcal{S}_A, C_1) = Q(M_1, C_1) = 0.90$ .

For  $C_2$ , which exits at  $M_2$ ,  $Q_{proj}(\mathcal{S}_A, C_2) = \text{Prob}_{pt}(M_1, C_2) \times Q(M_2, C_2) = 0.819$ .

For  $C_3$ , which exits at  $M_3$ ,  $Q_{proj}(\mathcal{S}_A, C_3) = \text{Prob}_{pt}(M_1, C_3) \times \text{Prob}_{pt}(M_2, C_3) \times Q(M_3, C_3) = 0.751$ .

For  $C_4$ , which exits at  $M_1$ ,  $Q_{proj}(\mathcal{S}_A, C_4) = Q(M_1, C_4) = 0.88$ .

The global projected quality is:

$$\sum_{j=1}^4 \text{Prob}(C_j) \cdot Q_{proj}(\mathcal{S}_A, C_j) = 0.844$$

The safety threshold is  $(1 - \epsilon) \times$  the role model’s global quality. With  $M_3$  having per-class qualities 0.98, 0.96, 0.96, and 0.97 for  $C_1$  through  $C_4$ :

$$(1 - 0.1) \times [0.4(0.98) + 0.3(0.96) + 0.2(0.96) + 0.1(0.97)] = 0.872$$

Since  $0.844 < 0.872$ ,  $\mathcal{S}_A$  is globally unsafe at  $\epsilon = 0.1$ . The longer chain causes too many  $C_3$  misclassifications, dragging down overall quality.

However, strategy  $\mathcal{S}_B = (M_1 \rightarrow M_3)$  requires events to pass through only one model before reaching  $M_3$  for non-exit classes, reducing the cumulative misclassification risk. For  $C_1$  and  $C_4$  that exit at  $M_1$ , projected qualities are 0.90 and 0.88. For  $C_2$  and  $C_3$  that exit at  $M_3$ , projected qualities are  $0.91 \times 0.96 = 0.874$  and  $0.85 \times 0.96 = 0.816$  respectively.

The global projected quality for  $\mathcal{S}_B$  is:

$$0.4(0.90) + 0.3(0.874) + 0.2(0.816) + 0.1(0.88) = 0.873$$

Since  $0.873 > 0.872$ ,  $\mathcal{S}_B$  is globally safe at  $\epsilon = 0.1$ . By using a shorter chain, NOMAD successfully constructs a strategy that maintains quality while reducing cost. This illustrates how the global safety check identifies viable strategies by balancing quality across the entire class distribution and preferring shorter chains when they suffice.

**4.2.1 Checking Global Chain Safety.** Algorithm 4 formalizes the global chain safety checking process. The algorithm forms a potential chain by appending the new model (line 1), then computes the global projected quality (lines 2-13) and checks if it meets the safety threshold (lines 14-15).

For each class (line 3), the algorithm computes the cumulative passthrough probability by multiplying the passthrough probabilities of all non-exit models (lines 6-12), identifies the exit model

(lines 7-10), and accumulates the weighted projected quality (line 13). The check on line 15 uses ‘<’ because it specifically tests for the unsafe condition: if the global projected quality is less than the threshold, the chain is unsafe and the function returns false. The chain is considered safe (returns true on line 16) only if the global projected quality exceeds the threshold.

Before a model is added to an event’s chain, this function is called to ensure the new, longer chain remains globally safe.

---

**Algorithm 4:** Check Global Chain Safety

---

**Input:**  $M_{new}, \mathcal{S}_{current}, M_r, \epsilon, C$   
**Output:** true if chain is safe, false otherwise

```

1  $S_{potential} \leftarrow \mathcal{S}_{current} \circ (M_{new});$  // Append new model
2  $Q_{global\_proj} \leftarrow 0;$ 
3 foreach class  $C_j \in C$  do
4    $Prob_{pass\_cum} \leftarrow 1.0;$ 
5    $M_{exit} \leftarrow \text{null};$ 
6   foreach model  $M_k \in S_{potential}$  do
7     if  $C_j \in EC(M_k)$  then
8        $M_{exit} \leftarrow M_k;$ 
9       break;
10    else
11       $Prob_{pass\_cum} \leftarrow Prob_{pass\_cum} \times Prob_{pt}(M_k, C_j);$ 
12      // Must pass through
13    if  $M_{exit} = \text{null}$  then
14       $M_{exit} \leftarrow M_r;$  // Fallback to RM
15     $Q_{global\_proj} \leftarrow Q_{global\_proj} + Prob(C_j) \times Prob_{pass\_cum} \times Q(M_{exit}, C_j);$ 
16   $Q_{threshold} \leftarrow (1 - \epsilon) \times \sum_{j=1}^{|C|} Prob(C_j) \times Q(M_r, C_j);$ 
17  if  $Q_{global\_proj} < Q_{threshold}$  then
18    return false; // Chain is unsafe
19 return true; // Chain is globally safe
```

---

**THEOREM 4.1.** A classification strategy  $\mathcal{S}$  constructed to satisfy the global chain safety condition meets the Global  $\epsilon$ -comparability requirement, if the estimated passthrough probabilities,  $Prob_{pt}^{est}$ , are less than or equal to the true probabilities  $Prob_{pt}^{true}$ .

**PROOF.** Let the realized quality of a strategy  $\mathcal{S}$  for class  $C_j$  be  $Q(\mathcal{S}, C_j)$ . By definition, this quality is the product of the true cumulative passthrough probability and the quality of the exit model,  $M_{exit}$ :

$$Q(\mathcal{S}, C_j) = \left( \prod_{M_k \text{ precedes } M_{exit}} Prob_{pt}^{true}(M_k, C_j) \right) \times Q(M_{exit}, C_j)$$

The projected quality is calculated using estimated probabilities:

$$Q_{proj}(\mathcal{S}, C_j) = \left( \prod_{M_k \text{ precedes } M_{exit}} Prob_{pt}^{est}(M_k, C_j) \right) \times Q(M_{exit}, C_j)$$

The theorem’s premise is that for any model,  $Prob_{pt}^{true}(M_k, C_j) \geq Prob_{pt}^{est}(M_k, C_j)$ . For relaxed estimation, this premise holds under the standard assumption that the validation set is representative of the true data distribution. Since probabilities are non-negative, the cumulative product also holds this inequality. Therefore, the

realized quality is greater than or equal to the projected quality:  $Q(\mathcal{S}, C_j) \geq Q_{proj}(\mathcal{S}, C_j)$ .

Multiplying both sides by the non-negative class probability  $Prob(C_j)$  and summing over all classes:

$$\sum_{j=1}^{|C|} Prob(C_j) \cdot Q(\mathcal{S}, C_j) \geq \sum_{j=1}^{|C|} Prob(C_j) \cdot Q_{proj}(\mathcal{S}, C_j)$$

The strategy is constructed to satisfy the global safety condition (Equation 7). Combining these inequalities yields:

$$\sum_{j=1}^{|C|} Prob(C_j) \cdot Q(\mathcal{S}, C_j) \geq (1 - \epsilon) \cdot \sum_{j=1}^{|C|} Prob(C_j) \cdot Q(M_r, C_j)$$

This proves the realized quality meets the global  $\epsilon$ -comparability requirement.  $\square$

The global chain safety mechanism with relaxed passthrough estimation provides an effective practical approach that balances quality guarantees with computational efficiency. Alternative approaches—including conservative passthrough estimation and class-based safety guarantees—are discussed in the appendix.

## 5 EXTENSIONS

This section describes two main extensions to the NOMAD framework that have a significant impact on our evaluation. The appendix also presents an alternative formulation of the model selection problem as a Markov Decision Process (MDP), in contrast to the utility-based greedy algorithm described here. The two approaches achieve comparable performance, with the MDP-based method showing only marginal improvements but at the cost of substantially higher complexity—particularly in dynamic settings that require costly retraining. For this reason, we focus in the main body on the greedy algorithm, while the MDP-based formulation and its comparison are discussed in the longer version.

**Extension to Dependent Models.** NOMAD extends to scenarios where model executions have dependencies, such as early-exit DNNs [27] (where intermediate classifiers depend on preceding layers) and stacking ensembles [8].

We represent inter-model relationships as a DAG  $G = (\mathcal{M}, E)$ , where edge  $(M_i, M_j) \in E$  signifies that  $M_j$  requires  $M_i$ . The prerequisite set  $\text{Prereq}(M_i)$  comprises all models that must execute before  $M_i$ . The core NOMAD loop adapts by tracking executed models  $\mathcal{M}_{exec}$  to determine ready models  $\mathcal{M}_{ready}$  at each iteration. A model becomes ready once all prerequisites are in  $\mathcal{M}_{exec}$ .

Since dependent models reuse computation from prerequisites, we redefine cost as **incremental cost**:

$$\text{inc\_cost}(M_i, \mathcal{M}_{exec}) = \text{cost}(M_i) - \sum_{M_j \in \text{Prereq}(M_i) \cap \mathcal{M}_{exec}} \text{shared\_cost}(M_j, M_i) \quad (8)$$

where  $\text{shared\_cost}(M_j, M_i)$  is the reused computation from  $M_j$ . The utility score becomes state-dependent:

$U(M_i, \mathcal{M}_{exec}) = \frac{\sum_{C_j \in EC(M_i)} P_{current}(C_j)}{\text{inc\_cost}(M_i, \mathcal{M}_{exec})}$ , ensuring models that leverage prior computation have higher utility.

*Batched Inference.* Our implementation processes events in **micro-batches** rather than individually. We use a timing-based batching approach where incoming events are accumulated until the batch reaches size  $N_{\text{batch}}$  or a timeout occurs (50ms). The batch is then processed through model chains with dynamic routing: after each model executes on the batch, events whose predicted class falls into an exit class terminate immediately, while remaining events update their beliefs and continue to the next selected model. This provides early exit benefits at the batch level—events classified by cheaper models terminate immediately, while only difficult events proceed to expensive models.

We use  $N_{\text{batch}} = 200$  as our default, which balances throughput and latency. Batching provides substantial speedups: 2-3 $\times$  for simple models (LDA, CART), 3-5 $\times$  for tree ensembles (RF, XGB), and 5-10 $\times$  for deep learning models through better vectorization and hardware utilization. NOMAD’s dynamic routing amplifies these benefits—because easier events exit early, expensive models only process the reduced subset of difficult events.

## 6 ADAPTIVE ALGORITHM

Our description of NOMAD has so far assumed a fixed class distribution. In real-world scenarios, however, this distribution can dynamically shift over time, a phenomenon known as **distributional drift**. To remain responsive to even sudden changes, NOMAD incorporates a mechanism to detect drift and adapt its class priors,  $Prob(C)$ , on an event-by-event basis. We deliberately choose lightweight statistical techniques for this task. While more complex machine learning models like LSTMs [15] or TimesFM [7] exist for time-series analysis, their computational overhead in an online setting could negate the very efficiency gains NOMAD is designed to provide while providing minimal forecasting accuracy gain. Our approach therefore prioritizes efficiency by using two classical statistical tools in tandem.

The first tool is the **Autoregressive Integrated Moving Average (ARIMA)** model[3], a time-series forecasting technique. We use a set of per-class ARIMA models to predict the probability of the next event belonging to a certain class based on the sequence of past events. The second tool is the **Page-Hinkley (PH) test**[22], a variant of CUSUM, a classic sequential analysis algorithm designed to detect abrupt changes in the average value of a data stream. We use it to monitor the performance of our ARIMA forecasts; a sudden drop in accuracy, flagged by the PH test, signals a potential distribution drift. The complete process is formalized in Algorithm 5.

The adaptive loop begins by using the ARIMA models to forecast the class distribution for the current event (Algorithm 5, Line 6). When the event’s true class,  $C_{\text{obs}}$ , is observed (Line 7), we quantify the model’s performance by calculating a “surprise” value: the negative log-likelihood of the observed class (Line 9). This stream of residual values is fed into the Page-Hinkley (PH) test (Line 10). The PH test tracks two running variables initialized to zero:  $m_t$ , a cumulative sum of the residuals adjusted by their running average, and  $M_t$ , the minimum value this sum has reached so far. A drift is flagged (Line 11) when the difference  $m_t - M_t$  exceeds a predefined threshold  $\lambda$ . Intuitively, this condition means that the model’s prediction errors are consistently increasing, which strongly suggests that the underlying class distribution has changed.

---

### Algorithm 5: Event-by-Event Adaptive Priors with ARIMA and Page-Hinkley Test

---

**Input:** Stream of new classified events  $\mathcal{E}$ ; PH test parameters: threshold  $\lambda$ , tolerance  $\delta$ ; Retraining buffer size  $N$ ;

```

1 Function AdaptivePriorUpdate()
2   Initialize ARIMA models  $\{A_i\}$  with historical data;
3   Initialize PH detector:  $ph \leftarrow (m = 0, M = 0)$ ;
4   Initialize event buffer  $\mathcal{B}$  (size  $N$ );
5   for each new classified event  $e$  in  $\mathcal{E}$  do
6      $Prob_{\text{pred}} \leftarrow$  Predict distribution for  $e$  using  $\{A_i\}$ ;
7      $C_{\text{obs}} \leftarrow$  Get class of event  $e$ ;
8     Add  $C_{\text{obs}}$  to buffer  $\mathcal{B}$ ;
9      $residual \leftarrow -\log(Prob_{\text{pred}}[C_{\text{obs}}])$ ;
10    Update  $ph$  with  $residual$  using PH update rule;
11    if  $ph$  signals a drift ( $m_t - M_t > \lambda$ ) then
12      Retrain ARIMA models  $\{A_i\}$  using events in  $\mathcal{B}$ ;
13      Reset PH detector  $ph \leftarrow (m = 0, M = 0)$ ;
14      // Adaptation triggered
15    Incrementally update  $\{A_i\}$  with  $C_{\text{obs}}$ ;
16     $Prob_{\text{current}}(C) \leftarrow$  Get forecast from updated  $\{A_i\}$ ;

```

---

This system employs a two-tier adaptation strategy to be both responsive and efficient. The first tier is a low-cost **incremental update** performed after every single event (Line 14). This step does not retrain the model from scratch. Instead, it feeds the latest observed class ( $C_{\text{obs}}$ ) and its corresponding forecast error back into the existing ARIMA model. This updates the model’s short-term memory of past observations (for its AutoRegressive component) and past errors (for its Moving Average component). This ensures the forecast for the very next event is always based on the most current information.

The second, more intensive tier is a full **retraining** (Line 12). This is triggered only when the PH test detects a significant drift, suggesting the underlying data patterns have fundamentally changed. In this case, the system re-estimates the core coefficients of the ARIMA models from scratch using the recent history of events stored in a buffer. This dual approach provides the benefits of immediate, event-by-event adjustments while reserving computationally expensive retraining for when it is truly necessary, thus maintaining the overall efficiency of NOMAD.

## 7 EVALUATION

We evaluate NOMAD across eight diverse datasets spanning tabular, image, and text modalities, comparing its cost-quality performance against baseline approaches. Our evaluation demonstrates NOMAD’s ability to achieve significant speedups while maintaining formal quality guarantees, examines its throughput advantages under realistic workloads, and validates its adaptive mechanisms under distribution drift. Below, we describe our experimental setup, present results, and analyze the findings.



## 7.1 Experimental Setup

**Datasets.** We selected eight publicly available datasets covering a diverse range of application domains, data modalities, and complexities. These include tabular data for network intrusion detection and activity recognition, image data for object recognition, and text data for sentiment analysis. This variety allows us to assess NOMAD’s generalizability.

Table 4 summarizes the datasets, listing for each dataset (a) the number of features, (b) number of classes, (c) a metric  $PS$  that quantifies the maximum achievable cost savings of any algorithm that selects a cheaper model while maintaining  $\epsilon$ -comparability to the role model, and (d)  $S_{\max}$ , which is the theoretically maximum speedup that can be achieved such an algorithm on the dataset. The value of  $PS$  and  $S_{\max}$  depend upon  $\epsilon$  and the role model (as will be clear below), and the reported values correspond to  $\epsilon = 0.1$  using the stacker as the role model. As  $\epsilon$  increases (i.e., as more quality degradation is tolerated), more cheap models qualify as  $\epsilon$ -comparable, both  $PS$  and  $S_{\max}$  increase accordingly.

$PS$  is computed by, for each validation event with true class  $c$ , selecting the cheapest model  $M_{c,cheap}$  that is  $\epsilon$ -comparable to the role model for that class. Such a selection can only be made by an **ideal oracle with perfect knowledge**. The cost saving is:

$$\text{Saving} = \frac{\text{cost}(M_r) - \text{cost}(M_{c,cheap})}{\text{cost}(M_r)}$$

$PS$  is the average of these savings across all validation events, weighted by the class distribution. Since  $PS$  is calculated based on the ideal oracle, it represents an **upper bound** on the savings attainable by any model selection algorithm. The **theoretical maximum speedup** ( $S_{\max}$ ) for a dataset can, thus, be computed as  $S_{\max} = 1/(1 - PS)$ . For example, UNSW-NB15 has  $PS = 0.74$ , meaning that an oracle could save 74% of the cost on average, corresponding to a maximum speedup of  $S_{\max} = 1/(1 - 0.74) = 3.85$ . Our goal in defining  $PS$  and  $S_{\max}$  is to determine how close NOMAD selection approach based on utility can approach the upper bound on savings/speedup.

**Experimental Hardware.** Our evaluation used a server-grade system (Intel Xeon 6706P-B Processor, 256 GB RAM, Ubuntu 22.04 LTS) with 8 CPU workers for parallel execution.

**Model Portfolio.** We trained a portfolio of 12-15 models per dataset using architectures appropriate for each data modality.

**Base Models:** For tabular datasets, we used traditional machine learning models including Linear Discriminant Analysis (LDA), Classification and Regression Trees (CART), K-Nearest Neighbors (KNN), Random Forest (RF), XGBoost (XGB), and Multi-layer Perceptrons (MLP) with varying depths. For image data (CIFAR-10), we employed Convolutional Neural Networks (CNNs) ranging from simple 3-layer architectures to deeper VGG-style and ResNet models. For text data (Twitter Sentiment), we used Logistic Regression on TF-IDF features, feedforward networks, and Transformer-based models like DistilBERT.

**Ensemble Models:** We trained two types of ensemble models: (1) *Stacking Ensembles*: Meta-models that combine predictions from multiple base models using Logistic Regression or XGBoost meta-learners. (2) *Early-Exit DNNs*: Neural networks with intermediate classification layers at different depths, allowing early termination for easy instances.

All models were trained to convergence on a training set and evaluated on a separate validation set to compute confusion matrices and per-class quality metrics. While the portfolio contains 12-15 models, *the number of possible model selection strategies (cascades) grows combinatorially*. NOMAD’s utility-based approach efficiently navigates this large space by selecting models dynamically based on class expectations rather than enumerating all possible cascades.

| Dataset                 | Feat. | Cls. | PS   | $S_{\max}$ |
|-------------------------|-------|------|------|------------|
| <i>Tabular Datasets</i> |       |      |      |            |
| UNSW-NB15 [1]           | 49    | 10   | 0.74 | 3.85       |
| CIC-IoT [4]             | 47    | 34   | 0.65 | 2.86       |
| ACI IoT [2]             | 44    | 8    | 0.78 | 4.55       |
| CIC-Flow [5]            | 79    | 10   | 0.81 | 5.26       |
| UCI HAR [30]            | 561   | 6    | 0.86 | 7.14       |
| ForestCover [29]        | 54    | 7    | 0.85 | 6.67       |
| <i>Image Dataset</i>    |       |      |      |            |
| CIFAR-10 [19]           | 3072  | 10   | 0.65 | 2.86       |
| <i>Text Dataset</i>     |       |      |      |            |
| Twitter Sentiment [10]  | Text  | 3    | 0.69 | 3.23       |

**Table 4: Overview of Datasets.**

**Baselines.** We compare NOMAD against two baseline strategies: (1) **Role Model Only**: Always executes the role model for every event in the batch. **Multi-Class Confidence Cascade**: We adapt the confidence-based cascade approach from [28, 35], which processes events sequentially through models ordered by increasing cost, exiting when a model’s prediction confidence exceeds a pre-defined threshold. We select this as our primary baseline because it represents the standard approach for cost-sensitive multiclass classification and shares NOMAD’s goal of using cheaper models when possible. Unlike training-time cascade methods [26, 36] that learn fixed cascade structures during training, confidence-based cascades can operate over arbitrary pretrained model portfolios, making them directly comparable to NOMAD. Note that neither training-time cascade methods nor confidence-based cascades provide formal quality guarantees—their confidence thresholds and learned boundaries are heuristic and do not ensure  $\epsilon$ -comparability to a role model. We implement this baseline using the same model portfolio available to NOMAD and tune the confidence threshold to achieve the best possible cost-quality tradeoff, representing the strongest possible configuration of this approach.

**Metrics and Evaluation Protocol.** We measure cost as average inference time per event on our evaluation hardware with batched execution. We measure quality using macro-averaged F1-score for overall performance and per-class F1-scores for granular analysis. The stacking ensemble achieved the highest F1-score across all datasets and serves as our default role model ( $M_r$ ). Our primary efficiency metric is speedup, calculated as the ratio of the Role Model’s cost to NOMAD’s cost. All results use 5-fold cross-validation with error bars showing  $\pm 1$  standard deviation.

**Default Configuration.** Unless stated otherwise, experiments use the following settings:  $\epsilon = 0.10$  (quality tolerance), stacker as role model, global safety mode with relaxed passthrough estimation, and batched inference with  $N_{\text{batch}} = 200$  as described in Section 5.

With batching, models achieve substantial throughput improvements: simple models (LDA, CART) process events at 0.9-3.3ms each, medium-complexity models (RF, XGB) at 3.9-6.2ms each, deep learning models at 8-15ms each, and ensemble models at 15-25ms each (per-event equivalent within batches).

## 7.2 Experiments

We begin with an experiment to verify that NOMAD ensures  $\epsilon$ -comparability and to demonstrate that alternative cascade approaches do not provide such guarantees. We then evaluate NOMAD’s cost-quality tradeoff and its impact on ingestion throughput, before examining its robustness under distribution drift.

**7.2.1 Quality Preservation.** Figure 2 shows that NOMAD successfully maintains the  $\epsilon$ -comparability guarantee across all datasets while the cascade baseline frequently violates quality constraints.

NOMAD achieves F1-scores between 0.91 and 1.03 times the role model’s F1-score (mean 0.96), staying well above the 0.90 threshold required by  $\epsilon = 0.10$ . Normalized scores above 1.0 indicate that NOMAD outperforms the role model; this occurs on some datasets (e.g., UCI HAR at 1.03) when cheaper models happen to perform better on frequently-occurring classes.

The cascade baseline, despite using the same models, achieves only 0.62-0.94 normalized quality. It violates the quality constraint on 6 out of 8 datasets because confidence scores do not reliably indicate per-class quality. High-confidence predictions can still be wrong, especially for minority classes.

This demonstrates NOMAD’s key advantage: formal quality guarantees through confusion matrix analysis rather than heuristic confidence thresholds.

**7.2.2 NOMAD Speedup.** To evaluate NOMAD’s performance, we compare the speedup achieved on different datasets against the theoretical maximum speedup,  $S_{\max}$ , that can be achieved by an oracle at the default setting ( $\epsilon = 0.10$ ).

Figure 2(b) shows NOMAD achieves speedups ranging from 2.1 $\times$  (CIFAR-10, CIC-IoT) to 6.4 $\times$  (UCI HAR), capturing 75-90% of the theoretical maximum speedup across all datasets. This demonstrates that NOMAD’s utility-based heuristic effectively approximates optimal model selection.

To evaluate how closely NOMAD approaches the theoretical optimum across different quality tolerances ( $\epsilon$ ), Figure 3 shows, for each dataset, the ratio between the speedup achieved by NOMAD and the maximum possible speedup ( $S_{\max}$ ) as  $\epsilon$  varies from 0.05 to 0.2. As illustrated, this ratio consistently remains between 0.70 and 0.95 across all  $\epsilon$  values, demonstrating that NOMAD delivers near-optimal performance throughout the tested range.

**7.2.3 Ingestion Throughput with NOMAD.** We next consider the impact on ingestion throughput under realistic workload conditions. Figure 4(a) demonstrates NOMAD’s throughput advantage by simulating an event stream with varying arrival rates for the UNSW-NB15 dataset using 8 parallel workers.

With batched execution, the role model baseline achieves 457 events/second while NOMAD achieves 1494 events/second—a 3.3 $\times$  improvement that matches the speedup ratio. This improvement comes from two synergistic effects:

**(1) Lower per-event cost:** NOMAD processes most events with cheap models (5.3ms average) versus the role model’s uniform high cost (17.5ms).

**(2) Asymmetric batching benefits:** NOMAD’s early-exit behavior creates an asymmetry that amplifies its advantage. Cheap models process full batches (200 events) for the majority of easy cases, maximizing parallelization efficiency. Expensive models only process residual mini-batches (20-40 events) of hard cases that couldn’t exit early. This means cheap models achieve near-optimal batching throughput on most events, while the baseline must always use expensive models with full batches on all events.

As arrival rates increase, both approaches eventually reach their thrashing points where queuing delays dominate. The role model thrashes at 457 events/s while NOMAD sustains up to 1494 events/s, providing substantial headroom for handling traffic spikes or reducing hardware requirements.

**7.2.4 Impact of Distribution Drift.** Real-world data streams often experience distribution shifts over time. User behavior changes, attack patterns evolve, and seasonal trends emerge. NOMAD’s utility-based model selection depends on class probabilities, so these shifts can degrade performance if left unaddressed. We evaluate NOMAD’s adaptive mechanism that detects distribution changes and updates its selection strategy.

We simulate an abrupt distribution shift in a stream of 10,000 events. The shift occurs at event 5,000, changing the class distribution significantly. Figure 4(b) shows the average cost per event over time, comparing adaptive and non-adaptive versions of NOMAD.

Before the shift, both versions perform identically at about 5.3ms per event (consistent with UNSW-NB15 batched performance). After the shift, the non-adaptive version immediately jumps to 7.4ms per event and stays there—a 40% cost increase. This happens because NOMAD continues using the old class distribution to make model selection decisions. The utility function prioritizes models that were optimal for the old distribution but are suboptimal for the new one.

The adaptive version also experiences an initial cost spike, but quickly recovers. The Page-Hinkley test detects the shift within about 200 events by monitoring the ARIMA forecast errors. Once detected, NOMAD retrains its distribution model using the 1,000 most recent events. Within 500 events after the shift, the adaptive version returns to optimal performance at 5.3ms per event.

The overhead of adaptation is minimal. Retraining takes only a few milliseconds per shift detection and occurs infrequently. Over the 5,000 post-shift events, the non-adaptive version incurs 10.5 extra seconds of cost (5,000 events  $\times$  2.1ms extra per event), while the adaptive version incurs only 2.6 extra seconds during recovery—a 4 $\times$  reduction in drift-related overhead.

**7.2.5 When Does Adaptation Help?** Figure 5(a) shows how the benefit of adaptation scales with drift intensity. We measure drift using KL divergence between the old and new distributions. At KL = 0 (no drift), there is no difference between adaptive and non-adaptive versions. As drift increases, adaptation prevents progressively more cost increase.

At moderate drift (KL = 0.4), adaptation prevents 20-35% cost increase across datasets. At severe drift (KL = 0.8), it prevents 30-50% increase. This shows that adaptation provides substantial value

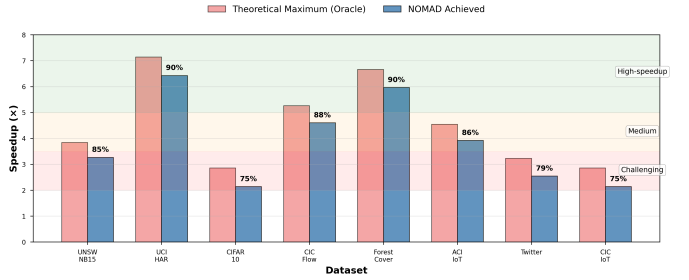
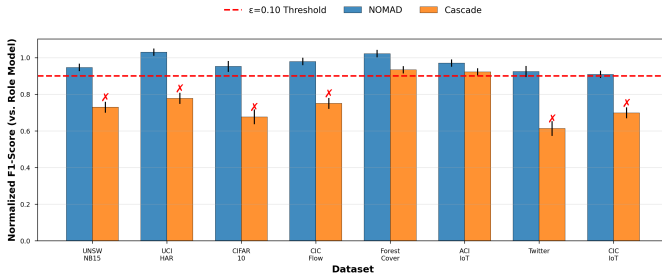


Figure 2: a) Quality comparison at  $\epsilon = 0.10$ . Error bars show  $\pm 1$  std dev. b) NOMAD speedup compared to  $S_{max}$  across datasets at  $\epsilon = 0.10$ .

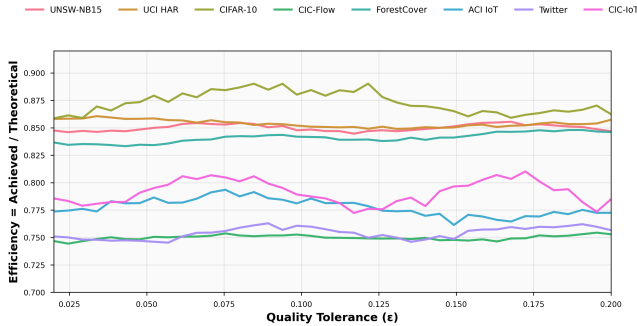


Figure 3: NOMAD Efficiency vs Oracle for varying  $\epsilon$  values.

in dynamic environments, with benefits growing as conditions become more volatile.

Some datasets benefit more from adaptation than others. CIC-IoT and ACI IoT show large benefits (45-52% at high drift) because their class distributions are imbalanced—shifts dramatically change which models are optimal. UCI HAR and ForestCover show smaller benefits (20-25%) because cheap models perform well across most distributions, limiting the impact of suboptimal selection.

**7.2.6 Tuning the Detection Threshold.** The Page-Hinkley test uses a threshold  $\lambda$  to control detection sensitivity. Figure 5(b) shows the tradeoff: lower  $\lambda$  detects shifts faster but may trigger false alarms on noisy data, while higher  $\lambda$  is more stable but slower to detect genuine shifts.

Across datasets, detection delay scales roughly linearly with  $\lambda$ . At  $\lambda = 100$  (our default), most datasets detect shifts within 140-180 events. At  $\lambda = 250$ , detection takes 310-450 events. The Twitter dataset is most sensitive to noise and benefits from higher  $\lambda$ , while stable datasets like UCI HAR can use lower  $\lambda$  for faster detection.

In practice,  $\lambda$  should be tuned based on the expected volatility of the data stream. For stable environments, use  $\lambda = 50 - 100$  for fast detection. For noisy or highly variable streams, use  $\lambda = 150 - 250$  to avoid false alarms. Our experiments use  $\lambda = 100$  as a reasonable default that balances detection speed and stability.

The retraining buffer size  $N$  (how many recent events to use for retraining) also affects performance. We use  $N = 1,000$  by default, which provides enough data to estimate the new distribution

accurately without including too much pre-drift data. Analysis of different  $N$  values is provided in the appendix.

### 7.3 Summary of Experimental Evaluation

Our evaluation demonstrates that NOMAD achieves 2-6 $\times$  speedups (50-84% cost reductions) while maintaining  $\epsilon$ -comparability guarantees, capturing 75-90% of the theoretical maximum performance across eight diverse datasets spanning tabular, image, and text modalities. NOMAD successfully preserves quality where confidence-based cascades fail, sustains 3-4 $\times$  higher ingestion throughput than always using the role model, and adapts to distribution shifts to prevent 20-50% cost increases that would otherwise occur. The framework’s benefits extend across traditional ML models, deep neural networks, ensemble methods, and dependent architectures like early-exit DNNs. The appendix includes additional experiments on sensitivity to initial class distribution estimates, detailed ablation studies of NOMAD’s components, alternative safety configurations (class-based vs. global, conservative vs. relaxed passthrough estimation), and deployment case studies with varying batch sizes and hardware configurations.

## 8 RELATED WORK

We discuss related work in database query processing, adaptive ML, and scalable data ingestion systems.

**Efficient Data Ingestion under Constraints.** Techniques like *batching* are commonly used to amortize overheads and improve ingestion throughput [11, 33]. Such techniques are complementary to NOMAD. NOMAD’s primary contribution lies in the intelligent selection and sequencing of models *after* ingestion, aiming to minimize computational cost per event while preserving classification quality. NOMAD can, however, be integrated with upstream batching mechanisms to improve ingestion performance further.

**Cost-Sensitive Classification.** Prior work such as cascade architectures [32] employ sequences of classifiers with increasing complexity, allowing early exits when samples are rejected with high confidence in binary tasks (e.g., face detection). NOMAD generalizes this to multi-class settings by dynamically selecting the next model based on utility, ensuring  $\epsilon$ -comparable accuracy across all classes. Related efforts include SpeedBoost [12], which modifies Adaboost to trade off accuracy and inference cost by factoring in weak learner costs, and early-exit deep neural networks (DNNs) [24, 27]

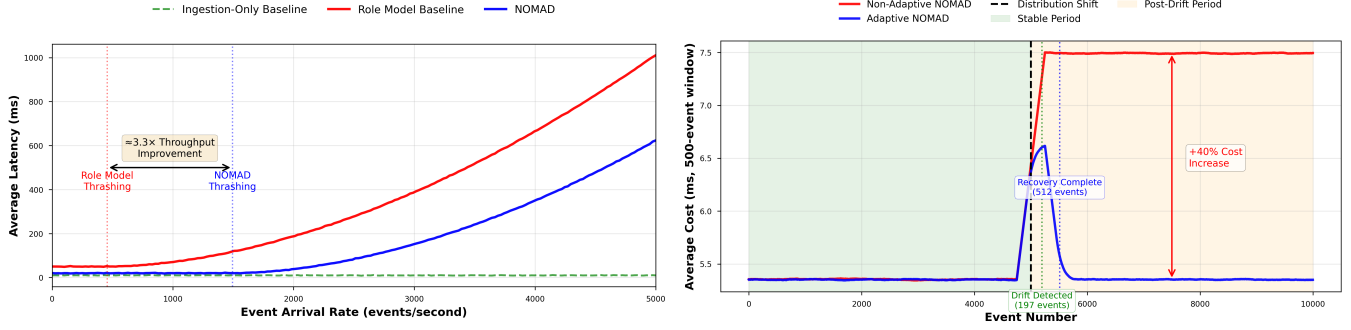


Figure 4: a) Throughput analysis on UNSW-NB15. b) Cost impact of distribution shift at 5000 events

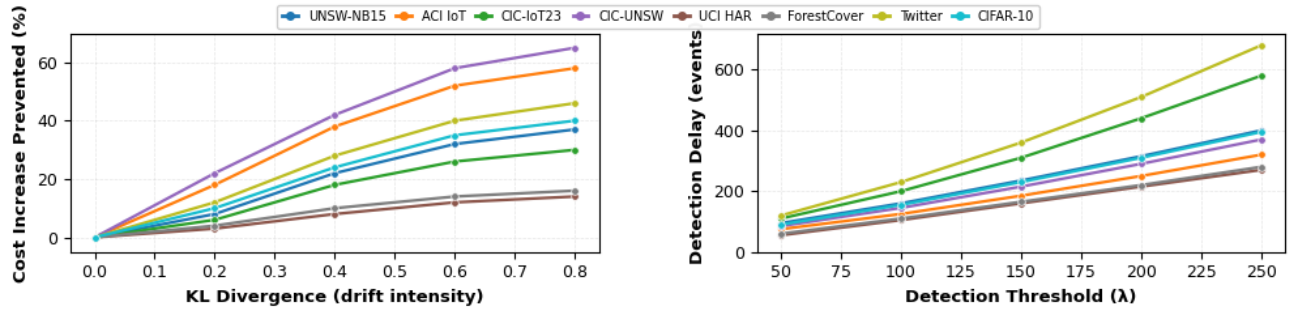


Figure 5: a) Adaptation benefit increases with drift intensity—at high drift ( $KL = 0.8$ ), adaptation prevents 30-50% cost increase across datasets. b) Detection threshold  $\lambda$  controls detection speed—lower values detect faster but may be sensitive to noise.

like BranchyNet [27], which add exit branches for early termination on "easy" inputs. In contrast to SpeedBoost, which tunes a single boosted classifier, and early-exit DNNs, that optimize a fixed monolithic network, NOMAD orchestrates chains of heterogeneous, pre-existing black-box models, selecting them dynamically via a utility function while maintaining classification quality.

**Predicate Ordering in Query Processing.** NOMAD shares conceptual similarities with predicate ordering in database query processing [14, 20], which reorders predicates by cost and selectivity so that inexpensive ones filter tuples before costlier ones execute<sup>1</sup>. Both use cheap operations as early filters to reduce downstream cost. However, the settings differ fundamentally: predicate ordering optimizes conjunctive queries where all predicates must hold, so a non-selective cheap predicate adds little overhead. In contrast, NOMAD addresses multiclass classification, where disagreement between cheap and expensive models negates savings since higher-cost models must still run to meet quality. Further, predicate ordering operates on boolean conditions, while NOMAD must explore a combinatorial space of model sequences over  $k > 2$  classes, considering class-specific performance and cost-quality tradeoffs.

**Adaptive Execution.** Adaptation has studied in both databases and ML to handle dynamic workloads and evolving data. In databases, techniques like adaptive indexing [6], cardinality estimation [21],

and storage tuning [17] monitor performance to adjust configurations. NOMAD adopts a similar strategy. Unlike database tuning, which may afford more complex models for offline tuning, NOMAD uses lightweight methods (e.g., ARIMA) to track prediction outcomes and update class probabilities in real time. In ML, adaptation techniques address concept drift in data streams [9, 34] through model retraining or reweighting. NOMAD focuses on detecting distribution drift to guide model selection, but does not yet adapt to model drift (e.g., degradation in a model's confusion matrix  $CM^{(i)}$ ). Supporting model drift adaptation is a promising direction for future work in dynamic settings like intrusion detection.

## 9 CONCLUSION

We introduced NOMAD, a framework for cost-efficient, real-time event classification that minimizes computational cost by intelligently sequencing classifiers while guaranteeing quality is  $\epsilon$ -comparable to a role model. By adapting to data distribution drift, NOMAD offers a robust and principled solution for stream processing, opening several avenues for future work - e.g., the classifier sequencing can be potentially modeled using Markov Decision Process (MDP) [16] instead of a greedy selection (though it opens up new challenge of integrating chain safety). Other challenges include supporting heterogeneous per-class quality constraints (varying  $\epsilon$  values), optimizing for multidimensional cost vectors (e.g., CPU, memory, power), and exploring the dual problem of maximizing classification quality under a fixed cost budget.

<sup>1</sup>Predicate ordering is conceptually related not only to NOMAD but also to cascade classifiers in general, and predates the latter by roughly a decade.

## REFERENCES

- [1] Australian Centre for Cyber Security (ACCS), University of New South Wales (UNSW). [n.d.]. ADFA-NB15 Datasets. <https://www.unsw.adfa.edu.au/australian-centre-for-cyber-security/cybersecurity/ADFA-NB15-Datasets/>. Accessed: 2025-04-03.
- [2] Nathaniel Bastian, David Bierbrauer, Morgan McKenzie, and Emily Nack. 2023. ACI IoT Network Traffic Dataset 2023. <https://doi.org/10.21227/qacj-3x32>
- [3] George E. P. Box, Gwilym M. Jenkins, Gregory C. Reinsel, and Greta M. Ljung. 2015. *Time Series Analysis: Forecasting and Control* (5th ed.). Wiley.
- [4] Canadian Institute for Cybersecurity (CIC), University of New Brunswick (UNB). [n.d.]. CIC IoT Dataset 2023. <https://www.unb.ca/cic/datasets/iotdataset-2023.html>. Accessed: 2025-04-03.
- [5] Canadian Institute for Cybersecurity (CIC), University of New Brunswick (UNB). [n.d.]. CIC UNSW-NB15 Augmented Dataset. <https://www.unb.ca/cic/datasets/cic-unswnb15.html>. Accessed: 2025-04-03.
- [6] Surajit Chaudhuri and Vivek Narasayya. 2007. Self-Tuning Database Systems: A Decade of Progress. In *Proceedings of the 33rd International Conference on Very Large Data Bases (VLDB '07)*. VLDB Endowment, 3–14.
- [7] Abhimanyu Das, Weihao Kong, Rajat Sen, and Yichen Zhou. 2024. A decoder-only foundation model for time-series forecasting. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*.
- [8] Thomas G Dietterich. 2000. Ensemble methods in machine learning. In *International workshop on multiple classifier systems*. Springer, 1–15.
- [9] João Gama, André Žliobaitė, Albert Bifet, Mykola Pechenizkiy, and Abdelhamid Bouchachia. 2014. A Survey on Concept Drift Adaptation. *Comput. Surveys* 46, 4, Article 44 (2014), 37 pages. <https://doi.org/10.1145/2523813>
- [10] Alec Go, Richa Bhayani, and Lei Huang. 2009. Twitter Sentiment Classification using Distant Supervision. In *CS224N Project Report, Stanford, Vol. 1*. 2009.
- [11] Raman Grover and Michael J. Carey. 2015. Data Ingestion in AsterixDB. In *International Conference on Extending Database Technology*. <https://api.semanticscholar.org/CorpusID:8057298>
- [12] Alex Grubb and J. Andrew Bagnell. 2012. SpeedBoost: A Computationally Efficient Algorithm for Cost-Sensitive Classification under Test-Time Budget. In *Proceedings of the 29th International Conference on International Conference on Machine Learning (ICML '12)*. Omnipress, Madison, WI, USA, 1047–1054.
- [13] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep Residual Learning for Image Recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 770–778. <https://doi.org/10.1109/CVPR.2016.90>
- [14] Joseph M Hellerstein and Michael Stonebraker. 1993. Predicate migration: Optimizing queries with expensive predicates. *ACM SIGMOD Record* 22, 2 (1993), 267–276.
- [15] Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long short-term memory. *Neural computation* 9, 8 (1997), 1735–1780.
- [16] Ronald A. Howard. 1960. *Dynamic Programming and Markov Processes*. The M.I.T. Press, Cambridge, MA.
- [17] Stratos Idreos, Olga Papaemmanouil, and Surajit Chaudhuri. 2015. Overview of Data Exploration Techniques. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data (SIGMOD '15)*. ACM, New York, NY, USA, 277–281. <https://doi.org/10.1145/2723372.2731084> (Slightly broader context, but reflects adaptive storage concepts Idreos worked on).
- [18] J. Kittler, M. Hatef, Robert P. W. Duin, and J. Matas. 1998. On Combining Classifiers. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 20, 3 (mar 1998), 226–239. <https://doi.org/10.1109/34.667881>
- [19] Alex Krizhevsky and Geoffrey Hinton. 2009. *Learning Multiple Layers of Features from Tiny Images*. Technical Report. University of Toronto.
- [20] Iosif Lazaridis and Sharad Mehrotra. 2007. Optimization of multi-version expensive predicates. In *Proceedings of the 2007 ACM SIGMOD International Conference on Management of Data (Beijing, China) (SIGMOD '07)*. Association for Computing Machinery, New York, NY, USA, 797–808. <https://doi.org/10.1145/1247480.1247568>
- [21] Guido Moerkotte, Thomas Neumann, and Gabriele Steidl. 2009. Preventing Bad Plans by Bounding the Impact of Cardinality Estimation Errors. In *Proceedings of the VLDB Endowment*, Vol. 2. VLDB Endowment, 982–993. <https://doi.org/10.14778/1687627.1687738>
- [22] E. S. Page. 1954. Continuous Inspection Schemes. *Biometrika* 41, 1/2 (1954), 100–115. <https://doi.org/10.2307/2333009>
- [23] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. 2021. Learning Transferable Visual Models From Natural Language Supervision. In *Proceedings of the 38th International Conference on Machine Learning (ICML '21) (Proceedings of Machine Learning Research)*, Vol. 139. PMLR, 8748–8763. <https://proceedings.mlr.press/v139/radford21a.html>
- [24] Haseena Rahmath P, Vishal Srivastava, Kuldeep Chaurasia, Roberto G. Pacheco, and Rodrigo S. Couto. 2024. Early-Exit Deep Neural Network - A Comprehensive Survey. *ACM Comput. Surv.* 57, 3, Article 75 (Nov. 2024), 37 pages. <https://doi.org/10.1145/3698767>
- [25] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. 2016. You Only Look Once: Unified, Real-Time Object Detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 779–788. <https://doi.org/10.1109/CVPR.2016.91>
- [26] Mohammad J Saberian and Nuno Vasconcelos. 2014. Multiclass Boosting: Theory and Algorithms. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 36, 3 (2014), 485–501. <https://doi.org/10.1109/TPAMI.2013.145>
- [27] Surat Teerapittayanon, Bradley McDanel, and H.T. Kung. 2016. BranchyNet: Fast inference via early exiting from deep neural networks. In *2016 23rd International Conference on Pattern Recognition (ICPR)*. IEEE, 2464–2469.
- [28] Kirill Trapeznikov and Venkatesh Saligrama. 2013. Supervised Sequential Classification Under Budget Constraints. In *Proceedings of the 16th International Conference on Artificial Intelligence and Statistics (AISTATS)*. 581–589.
- [29] UCI Machine Learning Repository. [n.d.]. Covertypes Data Set. <https://archive.ics.uci.edu/ml/datasets/Covertypes>. Accessed: 2025-04-03.
- [30] UCI Machine Learning Repository. [n.d.]. Human Activity Recognition Using Smartphones Dataset. <http://archive.ics.uci.edu/ml/datasets/Human+Activity+Recognition+Using+Smartphones>. Accessed: 2025-04-03.
- [31] Mesut Uğurlu and İbrahim Alper Doğru. 2019. A Survey on Deep Learning Based Intrusion Detection System. In *2019 4th International Conference on Computer Science and Engineering (UBMK)*. 223–228. <https://doi.org/10.1109/UBMK.2019.8907206>
- [32] Paul Viola and Michael J. Jones. 2004. Robust Real-Time Face Detection. *International Journal of Computer Vision* 57, 2 (2004), 137–154. <https://doi.org/10.1023/B:VISI.0000013087.49260.fb>
- [33] Xikui Wang and Michael J. Carey. 2019. An IDEA: an ingestion framework for data enrichment in asterixDB. *Proc. VLDB Endow.* 12, 11 (July 2019), 1485–1498. <https://doi.org/10.14778/3342263.3342628>
- [34] Gerhard Widmer and Miroslav Kubat. 1996. Learning in the Presence of Concept Drift and Hidden Contexts. *Machine Learning* 23, 1 (1996), 69–101. <https://doi.org/10.1007/BF00116900>
- [35] Xiaowei Xu, Eibe Frank, Bernhard Pfahringer, Geoffrey Holmes, and Quan Wang. 2014. Classifier selection for ensemble learning based on accuracy and diversity. *Pattern Recognition Letters* 51 (2014), 1–10.
- [36] Zhixiang Xu, Matt J Kusner, Kilian Q Weinberger, and Minmin Chen. 2014. Cost-Sensitive Tree of Classifiers. *Journal of Machine Learning Research* 15 (2014), 2321–2365.

## A CONSERVATIVE PASSTHROUGH ESTIMATION

The relaxed passthrough estimation method presented in Section 4.1 provides accurate estimates by using the full confusion matrix. However, in scenarios where stronger theoretical guarantees are desired or when the validation set may not be fully representative, a conservative estimation approach can be used.

### A.1 Conservative Estimation Method

We can estimate the passthrough probability conservatively by using the model’s recall. An input “passes through” if it is classified correctly or misclassified into a non-exit class. The probability of passing through is:

$$\text{Prob}_{pt}(M, C_j) = \text{Prob}(\text{correctly classified as } C_j) + \text{Prob}(\text{misclassified into a non-exit class}) \quad (9)$$

Here,  $\text{Prob}(\text{correctly classified as } C_j)$  is the recall,  $R(M, C_j)$ . The second term for safe misclassifications is always non-negative. We get a conservative estimate by using only the recall:

$$\text{Prob}_{pt}^{est} = R(M, C_j)$$

We consider this estimate conservative since it ignores any passthrough events from safe misclassifications. Like any validation-based metric, it is only reliable for real-world data if the validation set is representative.

## A.2 Comparison with Relaxed Estimation

The conservative method provides a strict lower bound on the true passthrough probability, which strengthens the theoretical guarantees of Theorem 4.2. However, this comes at a cost: by underestimating the passthrough probability, it results in overly pessimistic projected qualities, causing the safety check to reject chains that would actually be safe in practice.

As demonstrated in Appendix C, conservative estimation results in 15-25% lower speedups compared to relaxed estimation across all datasets. The difference is most pronounced on datasets where models frequently misclassify into non-exit classes (which are actually safe passthrough events), such as CIC-IoT and Twitter Sentiment.

For most practical applications, relaxed estimation provides the best balance of strong empirical performance with reasonable theoretical assumptions (that the validation set is representative). Conservative estimation may be preferred in safety-critical applications where absolute guarantees are paramount, even at the cost of reduced efficiency.

## B CLASS-BASED CHAIN SAFETY

While the global chain safety mechanism presented in Section 4 provides quality guarantees on average across all classes, some applications may require stricter per-class guarantees. The class-based chain safety approach ensures that the strategy’s quality for *every individual class* is  $\epsilon$ -comparable to the role model’s quality for that class. This provides the strongest guarantee but is more restrictive and may result in lower cost savings.

### B.1 Class-Based Safety Condition

For a chain to be class-based safe, its projected quality for every class must be  $\epsilon$ -comparable to the role model:

$$Q_{proj}(\mathcal{S}, C_j) \geq (1 - \epsilon) \times Q(M_r, C_j) \quad \forall C_j \in C \quad (10)$$

Equation 10 is satisfied for safe chains by construction: NOMAD only adds a model to a chain if the resulting chain passes the safety check in Algorithm 6. Any chain that violates this condition for any class is rejected (line 16 returns false), preventing its use. Thus, only chains satisfying the inequality for all classes are constructed.

**Example B.1.** Consider the chain  $\mathcal{S}_A = (M_1 \rightarrow M_2 \rightarrow M_3)$  with class-based recall as the quality metric  $Q$ . Let’s first set the tolerance  $\epsilon = 0.1$ . For an event of class  $C_2$ , it may be misclassified by  $M_1$  (with probability  $\text{Prob}_{mc}(M_1, C_2) = 0.12$ ) or pass through (with probability  $\text{Prob}_{pt}(M_1, C_2) = 0.88$ ). If it passes through, it will be classified by  $M_2$ , for which  $C_2$  is an exit class with quality  $Q(M_2, C_2) = 0.90$ . However, because the event could be incorrectly stopped by  $M_1$ , the chain’s effective quality for  $C_2$  is discounted:

$$Q_{proj}(\mathcal{S}_A, C_2) = \text{Prob}_{pt}(M_1, C_2) \times Q(M_2, C_2) = 0.88 \times 0.90 = 0.792$$

The safety threshold for class  $C_2$  is defined as  $(1 - \epsilon)$  times the role model’s quality:  $(1 - 0.1) \times Q(M_3, C_2) = 0.9 \times 0.96 = 0.864$ . Since  $0.792 < 0.864$ , the chain is unsafe for  $C_2$ .

If we relax the tolerance to  $\epsilon = 0.2$ , the threshold drops to  $(1 - 0.2) \times 0.96 = 0.768$ . Now,  $0.792 > 0.768$ , so the chain becomes safe for  $C_2$ .

However, let’s consider a class  $C_3$  event with  $\epsilon = 0.2$ . It must pass through  $M_1$  and then  $M_2$  to be classified by its exit model,  $M_3$ .

### Algorithm 6: Check Class-Based Chain Safety

---

**Input:**  $M_{new}, \mathcal{S}_{current}, M_r, \epsilon, C$   
**Output:** true if chain is safe, false otherwise

```

1  $\mathcal{S}_{potential} \leftarrow \mathcal{S}_{current} \circ (M_{new})$ ; // Append new model
2 foreach class  $C_j \in C$  do
3    $\text{Prob}_{pass\_cum} \leftarrow 1.0$ ;
4    $M_{exit} \leftarrow \text{null}$ ;
5   foreach model  $M_k \in \mathcal{S}_{potential}$  do
6     if  $C_j \in EC(M_k)$  then
7        $M_{exit} \leftarrow M_k$ ;
8       break;
9     else
10       $\text{Prob}_{pass\_cum} \leftarrow \text{Prob}_{pass\_cum} \times \text{Prob}_{pt}(M_k, C_j)$ ;
11      // Must pass through
12   if  $M_{exit} = \text{null}$  then
13      $M_{exit} \leftarrow M_r$ ; // Fallback to RM
14    $Q_{proj} \leftarrow \text{Prob}_{pass\_cum} \times Q(M_{exit}, C_j)$ ;
15    $Q_{thresh} \leftarrow (1 - \epsilon) \times Q(M_r, C_j)$ ;
16   if  $Q_{proj} < Q_{thresh}$  then
17     return false; // Chain unsafe for class  $C_j$ 
18 return true; // Chain is safe for all classes

```

---

Its effective quality is thus:

$$\begin{aligned} Q_{proj}(\mathcal{S}_A, C_3) &= \text{Prob}_{pt}(M_1, C_3) \times \text{Prob}_{pt}(M_2, C_3) \times Q(M_3, C_3) \\ &= 0.80 \times 0.92 \times 0.96 = 0.70656 \end{aligned}$$

Since 0.70656 is less than the safety threshold of 0.768 for  $C_3$ , the chain is still unsafe overall. While this specific chain is unsafe, an alternative strategy like  $\mathcal{S}_B = (M_1 \rightarrow M_3)$  can be shown to be safe for all classes with  $\epsilon = 0.2$ .

### B.2 Checking Class-Based Chain Safety

Algorithm 6 formalizes the class-based chain safety checking process. The algorithm forms a potential chain by appending the new model (line 1), then iterates over all classes (line 3). For each class, it computes the cumulative passthrough probability by multiplying the passthrough probabilities of all non-exit models (lines 6-12), identifies the exit model (lines 7-10), computes the projected quality (line 14), and checks if it meets the safety threshold (line 16). The check on line 16 uses ‘<’ because it specifically tests for the *unsafe* condition: if the projected quality is less than the threshold, the chain is unsafe and the function returns false. The chain is considered safe (returns true on line 18) only if it passes the safety check for all classes.

**THEOREM B.1.** *A classification strategy  $\mathcal{S}$  constructed to satisfy the class-based chain safety condition for all classes  $C_j \in C$  meets the Class-based  $\epsilon$ -comparability requirement, if the estimated passthrough probabilities,  $\text{Prob}_{pt}^{est}$ , are less than or equal to the true probabilities  $\text{Prob}_{pt}^{true}$ .*

**PROOF.** Let the realized quality of a strategy  $\mathcal{S}$  for class  $C_j$  be  $Q(\mathcal{S}, C_j)$ . By definition, this quality is the product of the true cumulative passthrough probability and the quality of the exit model,



$M_{exit}$ :

$$Q(S, C_j) = \left( \prod_{M_k \text{ precedes } M_{exit}} \text{Prob}_{pt}^{true}(M_k, C_j) \right) \times Q(M_{exit}, C_j)$$

The projected quality is calculated using estimated probabilities:

$$Q_{proj}(S, C_j) = \left( \prod_{M_k \text{ precedes } M_{exit}} \text{Prob}_{pt}^{est}(M_k, C_j) \right) \times Q(M_{exit}, C_j)$$

The theorem’s premise is that for any model,  $\text{Prob}_{pt}^{true}(M_k, C_j) \geq \text{Prob}_{pt}^{est}(M_k, C_j)$ . This premise holds based on the estimation method used: for conservative estimation (Appendix A), recall is by definition a lower bound on the true passthrough probability since it only counts correct classifications and ignores safe misclassifications into non-exit classes. For relaxed estimation, the premise holds under the standard assumption that the validation set is representative of the true data distribution. Since probabilities are non-negative, the cumulative product also holds this inequality. Therefore, the realized quality is greater than or equal to the projected quality:  $Q(S, C_j) \geq Q_{proj}(S, C_j)$ .

The strategy is constructed to satisfy the safety condition  $Q_{proj}(S, C_j) \geq (1 - \epsilon) \times Q(M_r, C_j)$ . Combining these inequalities yields:

$$Q(S, C_j) \geq Q_{proj}(S, C_j) \geq (1 - \epsilon) \times Q(M_r, C_j)$$

This proves that the realized quality meets the  $\epsilon$ -comparability requirement for class  $C_j$ . Since this holds for every class  $C_j \in C$ , the strategy meets the class-based  $\epsilon$ -comparability requirement.  $\square$

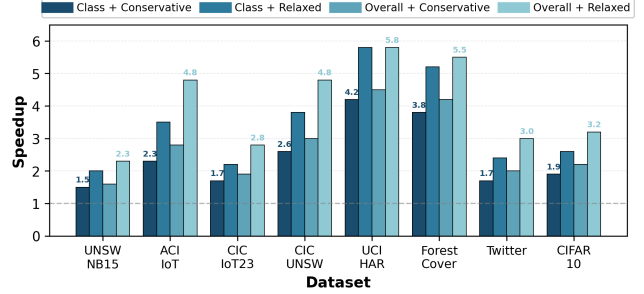
The class-specific constraint is more stringent than the global constraint, guaranteeing performance for every class including rare ones. The global constraint allows trading off performance across classes, potentially achieving lower costs but risking poor performance on minority classes. As shown in Appendix C, the global approach achieves significantly higher speedups (typically 0.5-1.5 $\times$  better) while still maintaining strong quality guarantees in practice.

## C EXPERIMENTAL COMPARISON OF SAFETY CONFIGURATIONS

While the main paper uses global chain safety with relaxed passthrough estimation as the default configuration, NOMAD’s framework supports alternative configurations that provide different tradeoffs between quality guarantees and computational efficiency. This section experimentally evaluates all four possible combinations of safety mechanisms and estimation methods.

### C.1 Configuration Options

NOMAD’s chain safety mechanism has two design dimensions. First, the safety scope can be either global (guarantees  $\epsilon$ -comparability on average across classes, allowing performance tradeoffs between classes) or class-based (guarantees  $\epsilon$ -comparability for every individual class, providing stronger per-class protection). Second, passthrough estimation can be either relaxed (uses the full confusion matrix to accurately estimate expected passthrough probability, as in Section 4.1) or conservative (uses only recall as a lower bound,



**Figure 6: Speedup comparison for all safety bound configurations ( $\epsilon = 0.10$ , stacker role model). The results form a clear ordering from most restrictive (Class-based + Conservative) to least restrictive (Global + Relaxed). Error bars show  $\pm 1$  standard deviation across 5-fold cross-validation.**

providing stronger theoretical guarantees at the cost of pessimistic estimates, as in Appendix A).

These dimensions yield four configurations: Global+Relaxed (the default used throughout the main paper), Global+Conservative, Class-based+Relaxed, and Class-based+Conservative.

### C.2 Experimental Setup

We evaluate all four configurations using the experimental setup from Section 7. All experiments use  $\epsilon = 0.10$ , the stacker as role model, and the same model portfolios and datasets. Figure 6 shows the speedup achieved by each configuration across all eight datasets.

### C.3 Results and Analysis

The results form a clear ordering from most restrictive (Class-based + Conservative) to least restrictive (Global + Relaxed), as shown in Figure 6.

Comparing Global vs. Class-based safety (holding estimation method constant), class-based safety consistently achieves 15-30% lower speedups across all datasets. This is expected: class-based safety must satisfy the  $\epsilon$  threshold for *every* class, including difficult minority classes where cheap models perform poorly. Global safety can compensate for weak performance on difficult classes by achieving stronger performance on easier, more frequent classes. The impact is most pronounced on imbalanced datasets like CIC-IoT and UNSW-NB15, where class-based safety prevents the use of cheap models on several minority classes despite their low frequency (and thus low impact on global quality). On more balanced datasets like UCI HAR and ForestCover, the difference is smaller (15-18%) since no single class dominates the distribution.

Comparing Relaxed vs. Conservative estimation (holding safety scope constant), conservative estimation achieves 10-20% lower speedups. Conservative estimation uses only recall as the passthrough probability, ignoring safe misclassifications into non-exit classes. This pessimistic estimate causes the safety check to reject chains that would actually maintain quality in practice. The impact varies by dataset characteristics. On datasets where models frequently misclassify into non-exit classes—such as CIC-IoT (20% speedup difference) and Twitter (18% difference)—conservative estimation is

overly pessimistic. On datasets where models tend to be either correct or exit-class-incorrect—such as UCI HAR (11% difference)—the gap is smaller.

The most restrictive configuration (Class-based + Conservative) achieves 35-45% lower speedups compared to the default (Global + Relaxed). For instance, on CIC-UNSW, the default achieves 4.2× speedup while Class-based+Conservative achieves only 2.4× speedup—a difference of 1.8× despite both maintaining the same  $\epsilon = 0.10$  quality guarantee.

Despite the speedup differences, *all four configurations successfully maintain their respective quality guarantees*. We verified this by measuring the realized quality on the test set. Global configurations maintain global  $\epsilon$ -comparability (weighted average quality  $\geq (1 - \epsilon) \times$  role model), class-based configurations maintain per-class  $\epsilon$ -comparability for every class, and all configurations meet their guarantees across all datasets.

## C.4 Configuration Recommendations

Based on these results, we recommend Global+Relaxed as the default, providing the best cost-quality tradeoff for most applications. Use this when quality averaged across classes is acceptable and when the validation set is reasonably representative. For applications requiring every individual class to meet quality requirements, such as in applications where minority classes represent critical but rare events (e.g., rare attack types in intrusion detection), use Class-based+Relaxed. For safety-critical applications where theoretical guarantees are paramount and efficiency is secondary, use Global+Conservative or Class-based+Conservative. Conservative estimation provides provable lower bounds on passthrough probability without assuming validation set representativeness.

The flexibility to choose among these configurations allows NO-MAD to adapt to different application requirements, balancing efficiency against the strength of quality guarantees.

## D DETAILS OF STATISTICAL METHODS

This appendix provides a more detailed technical overview of the two primary statistical methods used in our adaptive algorithm: the Autoregressive Integrated Moving Average (ARIMA) model for time-series forecasting and the Page-Hinkley (PH) test for change-point detection.

### D.1 Autoregressive Integrated Moving Average (ARIMA)

ARIMA is a powerful and widely used statistical model for analyzing and forecasting time-series data. It is a class of models that explains a given time series based on its own past values, that is, its own lags and the lagged forecast errors. The model’s name reflects its three core components: Autoregressive (AR), Integrated (I), and Moving Average (MA). An ARIMA model is typically denoted by the notation  $ARIMA(p, d, q)$ , where  $p$ ,  $d$ , and  $q$  are non-negative integers that specify the order of each component.

**AR: Autoregressive ( $p$ ).** The autoregressive component suggests that the value of the series at a given time  $t$ , denoted  $Y_t$ , can be modeled as a linear combination of its own past values. The parameter  $p$  is the order of the AR part, indicating how many lagged

observations are included in the model. A pure  $AR(p)$  model is expressed as:

$$Y_t = c + \sum_{i=1}^p \phi_i Y_{t-i} + \epsilon_t$$

where  $c$  is a constant,  $\{\phi_i\}$  are the model parameters (autoregressive coefficients), and  $\epsilon_t$  is a white noise error term at time  $t$ .

**I: Integrated ( $d$ ).** ARIMA models require the time series to be stationary, meaning its statistical properties such as mean and variance are constant over time. However, many real-world time series exhibit trends or seasonality and are therefore non-stationary. The "Integrated" component addresses this by applying differencing to the series. First-order differencing computes the change from one observation to the next:  $Y'_t = Y_t - Y_{t-1}$ . The parameter  $d$  is the order of differencing, representing the number of times the differencing operation is applied to the raw data to achieve stationarity.

**MA: Moving Average ( $q$ ).** The moving average component suggests that the value of the series at time  $t$  can be modeled as a linear combination of past forecast errors. The errors are the differences between the actual value and the forecast value at past time points. The parameter  $q$  is the order of the MA part, indicating how many lagged forecast errors are included in the model. A pure  $MA(q)$  model is expressed as:

$$Y_t = \mu + \epsilon_t + \sum_{j=1}^q \theta_j \epsilon_{t-j}$$

where  $\mu$  is the mean of the series,  $\{\theta_j\}$  are the model parameters (moving average coefficients), and  $\epsilon_t$  is the white noise error term.

An  $ARIMA(p, d, q)$  model combines these three components to model a non-stationary time series. The "fitting" process involves finding the optimal values for the parameters ( $p, d, q, \{\phi_i\}, \{\theta_j\}$ ) that best represent the data, often using methods like the Box-Jenkins methodology. Once fitted, an ARIMA model can be used to forecast future values of the series. Its computational efficiency, especially for forecasting and incremental updates, makes it a lightweight choice compared to more complex deep learning models for time-series analysis.

### D.2 Page-Hinkley (PH) Test

The Page-Hinkley (PH) test is a sequential analysis technique used for detecting a change or "drift" in the average of a signal. It is well-suited for online settings where data arrives in a stream, as it processes one data point at a time without needing to store the entire history.

The test works by monitoring a cumulative variable,  $m_T$ , which aggregates the differences between each observed data point and an expected mean, adjusted by a tolerance parameter. A drift is signaled when this cumulative variable increases by a significant amount, exceeding a predefined threshold.

The mathematical formulation is as follows. Given a stream of input values  $x_1, x_2, \dots, x_T$ , the PH test maintains two variables:

- (1) **The cumulative sum  $m_T$ :** This variable accumulates the evidence of a change. At each time step  $T$ , it is updated using the current observation  $x_T$ , its running mean  $\bar{x}_T$ , and



a magnitude parameter  $\delta$ .

$$m_T = \sum_{t=1}^T (x_t - \bar{x}_t - \delta)$$

Here,  $\delta$  represents the magnitude of change that is considered significant. It makes the test insensitive to small fluctuations, as deviations smaller than  $\delta$  will cause  $m_T$  to decrease.

- (2) **The running minimum  $M_T$ :** This variable tracks the minimum value the cumulative sum  $m_T$  has taken up to time  $T$ .

$$M_T = \min(m_t, t = 1 \dots T)$$

A drift is detected at time  $T$  if the difference between the current cumulative sum and its historical minimum exceeds a user-defined

threshold  $\lambda$ :

$$m_T - M_T > \lambda$$

The parameter  $\lambda$  controls the sensitivity of the test. A smaller  $\lambda$  allows for faster detection of drifts but may lead to more false alarms. Conversely, a larger  $\lambda$  makes the test more robust to noise but slower to react to a genuine change.

In the context of our system, the input stream  $\{x_t\}$  is the sequence of residuals from our ARIMA forecasts. A stable data distribution results in small, randomly fluctuating residuals. When a drift occurs, the ARIMA models become less accurate, causing a persistent increase in the average residual. The PH test is designed to detect exactly this type of upward shift in the mean of the residual signal, providing a lightweight and reliable trigger for model adaptation.